پنل مدیریت Persona Path

در این گزارش به شرح مختصر کد پنل مدیریت می پردازیم. ابتدا ابزار های استفاده شده، سپس ساختار کد پروژه و در انتها منطق کلی هر بخش توضیح داده شده است.

ابزار ها:

۱. Next.js:

Next.js یک قالب (framework) منبع باز محبوب است که بر روی React.js ساخته شده است و برای ساخت برنامه های کاربردی وب مدرن طراحی شده است. این ابزار مجموعه‌ای از ویژگی‌ها و قراردادهای قدرتمند را برای ساده‌سازی فرآیند توسعه و افزایش بهره‌وری در اختیار توسعه‌دهندگان قرار می‌دهد.

Next.js قابلیت‌های ارائه سمت سرور (SSR-Server Side Rendering)، تولید سایت ایستا (SSG-Static Site Generation) و ارائه سمت مشتری (CSR-Client Side Rendering) را ارائه می‌دهد که آن را برای طیف وسیعی از موارد استفاده، از وب‌سایت‌های ایستا ساده گرفته تا برنامه‌های سازمانی پیچیده، مناسب می‌سازد.

هدف و کاربرد ابزار:

Next.js توسعه برنامه های کاربردی وب مبتنی بر React را با ارائه یک رویکرد ساختاریافته برای ایجاد رابط های کاربری ساده می کند. این ابزار یکپارچه با React ادغام می شود و ویژگی‌هایی علاوه بر React، مانند تقسیم خودکار کد، مسیریابی بهینه، و پشتیبانی داخلی CSS و Sass را ارائه می دهد.

توسعه‌دهندگان می‌توانند از Next.js برای ایجاد برنامه‌های کاربردی وب سریع، کارآمد و سئو پسند با حداقل هزینه پیکربندی پروژه خود استفاده کنند. سئو (Search Engine Optimization)، عمل بهینه سازی یک وب سایت برای بهبود دید و رتبه آن در صفحات نتایج موتورهای جستجو است.

ویژگی های کلیدی:

- **رندر سمت سرور (SSR)**: Next.js به توسعه دهندگان این امکان را می دهد تا مولفه های React را در سمت سرور رندر کنند که عملکرد را بهبود می بخشد، و با ارائه HTML کاملاً رندر شده به مشتریان، بهینه سازی سئو را نیز تسهیل می کند.

  چگونه کار می کند: وقتی کاربر یک صفحه وب را درخواست می کند، سرور محتوای HTML را به صورت پویا تولید می کند، از جمله هر داده ای که از پایگاه داده یا API های خارجی واکشی شده است. سپس سرور این HTML از پیش رندر شده را برای مشتری ارسال می کند، که می تواند بلافاصله محتوا را به کاربر نمایش دهد.

  مزایا: SSR معمولاً منجر به زمان بارگذاری اولیه صفحه سریع‌تر و عملکرد سئوی بهتر می‌شود، زیرا موتورهای جستجو می‌توانند محتوای HTML را به طور مؤثرتری خزیده (crawl) و فهرست‌بندی کنند.

- **تولید سایت ایستا (SSG):** Next.js از تولید سایت ایستا پشتیبانی می کند و به توسعه دهندگان این امکان را می دهد تا صفحات را در زمان ساخت پیش رندر کنند و آنها را به عنوان فایل های HTML ایستا ارائه کنند. این رویکرد عملکرد را افزایش می دهد و بار سرور را کاهش می دهد، به ویژه برای وب سایت های پر محتوا.

  چگونه کار می کند: در طول فرآیند ساخت، سرور صفحات HTML را برای هر مسیر یا صفحه وب سایت تولید می کند. سپس این فایل‌های HTML به صورت ایستا ذخیره می‌شوند و در صورت درخواست، بدون نیاز به پردازش سمت سرور، به مشتریان ارائه می‌شوند.

  مزایا: SSG عملکرد و مقیاس‌پذیری عالی ارائه می‌دهد، زیرا فایل‌های HTML را می‌توان به طور مستقیم در یک شبکه تحویل محتوا (CDN- Content Delivery Network) در حافظه پنهان ذخیره کرد و استفاده کرد. همچنین الزامات زیرساخت سرور را ساده می کند و بار سرور را کاهش می دهد.

- **رندر سمت مشتری (CSR):** Next.js از رندر سمت کلاینت هم برای تجربه پویا کاربر پشتیبانی می‌کند، جایی که محتوا با استفاده از جاوا اسکریپت در سمت کلاینت واکشی و رندر می‌شود، البته معمولاً پس از بارگیری محتوای اولیه HTML..

  چگونه کار می کند: هنگامی که یک کاربر از یک صفحه وب بازدید می کند، سرور یک سند پایه HTML را همراه با کد جاوا اسکریپت ارسال می کند. کد جاوا اسکریپت در مرورگر اجرا می شود، داده ها را از API ها واکشی می کند و محتوای صفحه را به صورت پویا ارائه می دهد.

  مزایا: CSR تجربیات کاربر بسیار پویا و تعاملی را امکان پذیر می کند، زیرا محتوا را می توان بدون بارگیری مجدد کامل صفحه به روز کرد. همچنین امکان واکشی و ارائه داده های انعطاف پذیرتر را فراهم می کند.

- **تقسیم خودکار کد:** Next.js به طور خودکار بسته های جاوا اسکریپت را بر اساس مرزهای صفحه تقسیم می‌کند که زمان بارگذاری را بهینه می کند و عملکرد کلی را بهبود می بخشد.

- **مسیریابی بهینه:** Next.js یک سیستم مسیریابی ساده و بصری را ارائه می دهد که به توسعه دهندگان این امکان را می دهد تا مسیرها را با استفاده از قراردادهای مسیریابی مبتنی بر فایل تعریف کنند، کد های اضافی مسیریابی را کاهش دهند و سازماندهی کد را بهبود بخشند.

- **پشتیبانی داخلی CSS و Sass:** پشتیبانی داخلی از ماژول‌های CSS و Sass از ویژگی های Next.js است که به توسعه‌دهندگان اجازه می‌دهد تا صفحات ماژول بندی شده و قابل نگهداری بالایی را بسازند.

جمع‌بندی ابزار Next.js:

Next.js به دلیل مجموعه ویژگی‌های متعدد و قوی، قابلیت های بهینه سازی عملکرد و پشتیبانی فعال جامعه، به عنوان قالب توسعه اولیه برای پنل انتخاب شد. با استفاده از Next.js، ما توانستیم توسعه را

تسریع کنیم، قابلیت نگهداری کد را بهبود بخشیم، و یک تجربه کاربری یکپارچه را برای کاربران پنل ارائه کنیم. در حالی که Next.js ممکن است محدودیت‌های خاصی را تحمیل کند، مزایای آن بیشتر از معایب است و آن را به یک انتخاب قانع‌کننده برای ساخت برنامه‌های کاربردی وب مدرن تبدیل می‌کند.

۲. MUI (Meterial-UI) :

این ابزار یک قالب محبوب React UI منبع باز است که مجموعه‌ای از اجزای از پیش طراحی‌شده و قابل تغییر را با پیروی از دستورالعمل‌های Google Material Design در اختیار توسعه‌دهندگان قرار می‌دهد. طیف وسیعی از مؤلفه‌ها (components)، استایل ها و ابزارهای کاربردی را برای ساده‌سازی توسعه رابط‌های کاربری مدرن و جذاب ارائه می‌دهد.

هدف و کاربرد ابزار:

MUI با ارائه یک کتابخانه جامع از مولفه ها و استایل های قابل استفاده مجدد، فرآیند ساخت رابط های کاربری انعطاف پذیر (responsive) و زیبا را ساده می کند. توسعه‌دهندگان می‌توانند از MUI برای ایجاد طرح‌های رابط کاربری سازگار و بصری که با اصول طراحی متریال (Material Design) مطابقت دارند، استفاده کنند، و در نتیجه یک تجربه کاربری حرفه‌ای ایجاد کنند.

ویژگی های کلیدی:

- **مولفه های از پیش طراحی شده:** MUI مجموعه ای غنی از مولفه های از پیش طراحی شده، شامل دکمه ها، فیلد های ورودی، دیالوگ ها و موارد دیگر را ارائه می دهد که توسعه دهندگان می توانند به راحتی آنها را در برنامه های خود قرار دهند. این اجزا به گونه ای طراحی شده اند که انعطاف پذیر و قابل تغییر باشند و به توسعه دهندگان این امکان را می دهند تا آنها را متناسب با نیازهای طراحی خاص خود تطبیق دهند.

- **تم بندی و استایل‌سازی:** MUI قابلیت‌های تم و استایل سازی قوی را ارائه می‌دهد و به توسعه‌دهندگان اجازه می‌دهد ظاهر اجزا را مطابق با ترجیحات طراحی خود شخصی سازی کنند. توسعه‌دهندگان می‌توانند تم‌ها، رنگ‌ها، تایپوگرافی و گزینه‌های فاصله‌گذاری شخصی سازی شده را برای ایجاد طرح‌های UI منسجم تعریف کنند.

- **طراحی انعطاف پذیر:** اجزای MUI با در نظر گرفتن انعطاف پذیری (responsive) طراحی شده اند و اطمینان حاصل می کنند که به طور یکپارچه با اندازه های صفحه نمایش و دستگاه های مختلف سازگار می شوند و تجربه کاربری ثابتی در سراسر دسکتاپ، تبلت و دستگاه‌های تلفن همراه برای کاربران فراهم می کنند.

جمع‌بندی ابزار MUI:

Material-UI در شکل‌دهی پنل ما نقش بسزایی داشته است، و مجموعه‌ای از مولفه های قوی، استایل ها و برنامه‌های کاربردی را ارائه داد که به طور قابل‌توجهی روند توسعه رابط کاربری ما را تسریع کرده‌اند. ما توانستیم یک رابط کاربری جذاب و منسجم از نظر بصری ایجاد کنیم که به طور یکپارچه در دستگاه‌های

مختلف عمل کند و ظاهر زیبایی داشته باشد. مزایای بی شمار MUI از نظر بهره وری، سازگاری طراحی و دسترسی بالا بسیار قابل توجه تر از چالش های استفاده از آن و معایبش بود. MUI به عنوان یک متحد ضروری در تلاش ما برای ارائه یک تجربه کاربری برتر ظاهر شده است و ما را قادر می سازد تا رابط های کاربر پسند ایجاد کنیم که با مخاطبان ما سازگار باشد.

۳. Axios :

Axios یک کتابخانه محبوب جاوا اسکریپت است که برای ایجاد درخواست های HTTP از مرورگر و محیط های Node.js استفاده می شود. این برنامه یک API ساده را برای انجام عملیات ناهمگام (asynchronous) مانند واکشی داده ها از APIها، ارسال داده ها به سرورها و مدیریت پاسخ ها در اختیار توسعه دهندگان قرار می دهد. Axios به دلیل سهولت استفاده، انعطاف پذیری و مجموعه ویژگی های قوی دیگر، به طور گسترده در پروژه های توسعه وب مدرن مورد استفاده قرار می گیرد.

هدف و کاربرد ابزار:

Axios با ارائه یک API ساده و پشتیبانی داخلی از ویژگی‌هایی مانند رهگیری درخواست و پاسخ، لغو خودکار درخواست و مدیریت خطا، فرآیند ایجاد درخواست‌های HTTP در برنامه‌های وب را ساده می‌کند. توسعه دهندگان می توانند از Axios برای برقراری ارتباط با سرورهای بک اند (back-end)، استفاده از API های RESTful و انجام عملیات CRUD (ایجاد، خواندن، به روز رسانی، حذف) بدون نیاز به تنظیمات پیچیده یا رسیدگی دستی به درخواست های HTTP استفاده کنند.

ویژگی های کلیدی:

- **API مبتنی بر Promise**: اکسیوس از promise های جاوا اسکریپت برای مدیریت عملیات های ناهمگام (asynchronous) استفاده می‌کند و به توسعه‌دهندگان اجازه می‌دهد کدهای تمیز و مختصر بنویسند که خواندن و نگهداری آن آسان باشد. Promise ها به توسعه‌دهندگان این امکان را می‌دهد تا چندین عملیات ناهمگام را با هم زنجیره‌ای کنند و شرایط موفقیت و خطا را به شیوه‌ای ثابت مدیریت کنند.

  توضیح Promise ها : یک مفهوم اساسی در جاوا اسکریپت برای مدیریت عملیات های ناهمگام است. promises نشان دهنده یک مکان نگهدار (placeholder) برای نتیجه یک عملیات ناهمگام هستند. عملیات های ناهمگام در جاوا اسکریپت اجازه می دهند تا وظایف مستقل از جریان اصلی برنامه اجرا شوند، رفتار غیر مسدود کننده در اجرا را ممکن می کنند و پاسخگویی را در برنامه های وب بهبود می‌بخشند، چرا که اجرا منتظر یک قسمت به‌خصوص که کندتر از بقیه است نمی‌شود و از آن عبور می‌کند (رفتار غیر مسدود کننده) و هر زمان آن قسمت به پایان رسید اجرای آن را به صورت مجزا انجام می دهد. نتیجه عملیات های ناهمگام ممکن است در حال انتظار باشد، با یک مقدار تکمیل شده، یا با یک خطا رد شده باشد.

  Promise ها راهی برای کار با کدهای ناهمزمان به شیوه‌ای ساختاریافته‌تر و قابل مدیریت‌تر فراهم می‌کند و به توسعه‌دهندگان اجازه می‌دهد تا کدهای تمیزتر، خواناتر و کمتر مستعد خطا بنویسند.

نحوه کار Promsie ها:

هنگامی که یک تابع یک promise را برمی گرداند، به این معنی است که تابع در نهایت نتیجه ای را، چه با موفقیت یا ناموفق، تولید خواهد کرد. promise ها سه حالت دارند:

- **در حال انتظار (Pending)**: حالت اولیه یک promsie. این نشان می دهد که عملیات ناهمگام مرتبط با promise هنوز کامل نشده است.
- **اتمام یافته (Fulfilled)**: حالت یک promise زمانی که عملیات ناهمگام با موفقیت کامل می شود و یک مقدار در دسترس است.
- **رد شده (Rejected)**: حالت یک promise زمانی که عملیات ناهمگام با خطا مواجه می شود یا نمی تواند مقداری تولید کند.

توسعه‌دهندگان می‌توانند با استفاده از روش .then() برای رسیدگی به نتایج موفقیت‌آمیز و متد catch() برای رسیدگی به خطاها، به نتایج promise ها دسترسی پیدا کنند. این اجازه می دهد تا منطق ترتیبی و مدیریت خطا به خوبی سازماندهی شده و از جریان اصلی کد جدا شود.

فواید Promise ها:

Promise ها روشی ساختاریافته و متوالی برای مدیریت عملیات ناهمزمان ارائه می‌کند، که استدلال و حفظ کد ناهمزمان را آسان‌تر می‌کند.آنها همچنین از روش زنجیره‌ای پشتیبانی می‌کند، که اجازه می‌دهد چندین عملیات ناهمزمان به‌طور متوالی یا موازی اجرا شوند. این قابلیت استفاده مجدد و ماژولار بودن کد را ارتقا می دهد.

علاوه بر موارد قبل، Promise ها اجتناب از جهنم پاسخ ها (callback hell) را با ارائه روشی قابل مدیریت پذیرتر از callback ها، کاهش تودرتو بودن و بهبود وضوح کد امکان پذیر می کنند.

- **رهگیری درخواست و پاسخ HTTP**: اکسیوس قلاب‌هایی (hooks) را برای رهگیری درخواست‌ها و پاسخ‌های HTTP فراهم می‌کند و به توسعه‌دهندگان اجازه می‌دهد تا سرصفحه‌های (headers) درخواست را تغییر دهند، داده‌های درخواست و پاسخ را تغییر دهند و منطق مدیریت خطای جهانی را پیاده‌سازی کنند. این ویژگی، انعطاف‌پذیری و شخصی سازی را افزایش می‌دهد و توسعه‌دهندگان را قادر می‌سازد تا Axios را با موارد استفاده و نیازهای خاص خود تطبیق دهند.

- **لغو خودکار درخواست**: Axios از لغو خودکار درخواست های معلق پشتیبانی می کند و از ترافیک غیرضروری شبکه و مصرف منابع جلوگیری می کند. توسعه دهندگان می توانند درخواست ها را به صورت دستی یا خودکار بر اساس شرایط از پیش تعریف شده لغو کنند و عملکرد و کارایی را در سناریوهایی مانند صفحه بندی بهبود بخشند.

- **مدیریت خطا**: Axios قابلیت‌های مدیریت خطای جامعی را ارائه می‌دهد و به توسعه‌دهندگان این امکان را می‌دهد تا خطاهای شبکه، کدهای وضعیت HTTP و سایر شرایط خطا را به خوبی مدیریت کنند. پاسخ های خطا به طور خودکار تجزیه و به اشیاء خطای جاوا اسکریپت تبدیل می

شوند و اجرای استراتژی های بازیابی خطا و ارائه پیام های خطای آموزنده به کاربران را آسان می کند.

مقایسه با سایر ابزارها:

Axios با سایر کتابخانه های سرویس گیرنده HTTP مانند Fetch API و jQuery.ajax () رقابت می کند. در حالی که Fetch API یک ویژگی داخلی مرورگرهای مدرن است و یک API مبتنی بر promise مشابه به Axios ارائه می‌کند، Axios ویژگی‌های اضافی مانند لغو درخواست، رهگیری درخواست و پاسخ، و تجزیه خودکار JSON را ارائه می‌کند. علاوه بر این، Axios نسبت به Fetch API دارای یک API بصری تر و سازگارتر است که آن را به انتخابی ارجح برای بسیاری از توسعه دهندگان تبدیل می کند. در مورد jQuery.ajax()، در حالی که عملکردی مشابه با Axios ارائه می دهد، به کتابخانه jQuery گره خورده است و ممکن است مانند Axios سبک یا مدرن نباشد.

جمع‌بندی ابزار Axios:

Axios به عنوان یک ابزار ضروری در توسعه وب مدرن ظاهر شده است و به توسعه دهندگان راه حلی قدرتمند و همه کاره برای ایجاد درخواست های HTTP و مدیریت عملیات های ناهمگام ارائه می دهد. Axios با سادگی، انعطاف‌پذیری و مجموعه ویژگی‌های دیگر قوی خود، فرآیند برقراری ارتباط با سرورها و استفاده از APIها را ساده می‌کند و به توسعه‌دهندگان اجازه می‌دهد تا برنامه‌های وب کارآمد و قابل اعتماد بسازند.

۴. Docker :

Docker یک پلتفرم محبوب برای توسعه، حمل و اجرای برنامه‌های کاربردی با استفاده از فناوری کانتینرسازی است. این به توسعه دهندگان اجازه می دهد تا برنامه ها و وابستگی‌های آن را در کانتینرهای سبک و قابل حمل بسته بندی کنند که می توانند به طور مداوم در محیط های مختلف اجرا شوند. داکر ابزارها و گردش کارها را برای ساخت، توزیع و مدیریت کانتینرها فراهم می کند و فرآیند توسعه و استقرار برنامه های نرم افزاری را ساده می کند.

هدف و کاربرد ابزار:

Docker فرآیند استقرار برنامه‌ها را با محصور کردن آنها در کانتینرها، که شامل تمام وابستگی‌ها و کتابخانه‌های لازم برای اجرای برنامه است، ساده می‌کند. توسعه‌دهندگان می‌توانند یک Dockerfile را تعریف کنند، که حاوی دستورالعمل‌هایی برای ساختن یک تصویر Docker است که به‌عنوان طرحی برای کانتینر عمل می‌کند. سپس تصویر Docker می‌تواند بر روی هر پلتفرمی که از Docker پشتیبانی می‌کند توزیع و اجرا شود و از رفتار ثابت در محیط‌های مختلف اطمینان حاصل کند.

ویژگی های کلیدی:

- **Containerization**: داکر از فناوری کانتینر سازی برای جداسازی برنامه ها و وابستگی های آنها در محیط های سبک وزن و قابل حمل استفاده می کند. هر کانتینر به‌عنوان یک فرآیند مستقل

اجرا می‌شود، با سیستم فایل، شبکه و محیط خاص خود، سازگاری و تکرارپذیری را در محیط‌های استقرار مختلف فراهم می‌کند.

- **Dockerfile**: یک اسکریپت مبتنی بر متن است که مراحل ساخت یک تصویر داکر را مشخص می کند. این شامل دستورالعمل‌هایی برای نصب وابستگی‌ها، کپی کردن فایل‌ها، پیکربندی متغیرهای محیطی و تعیین محیط زمان اجرا برای برنامه است. Dockerfile به توسعه دهندگان اجازه می دهد تا فرآیند کانتینری سازی را خودکار کرده و از سازگاری بین محیط های توسعه، آزمایش و تولید اطمینان حاصل کنند.

- **Docker Image**: تصویر Docker یک بسته سبک وزن، مستقل و قابل اجرا است که شامل همه چیزهایی است که برای اجرای یک کانتینر داکر لازم است، از جمله کد برنامه، زمان اجرا، کتابخانه ها و وابستگی ها. تصاویر از Dockerfiles ساخته شده‌اند و می‌توانند نسخه‌بندی شوند، برچسب‌گذاری شوند و از طریق رجیستری های Docker مانند Docker Hub به اشتراک گذاشته شوند.

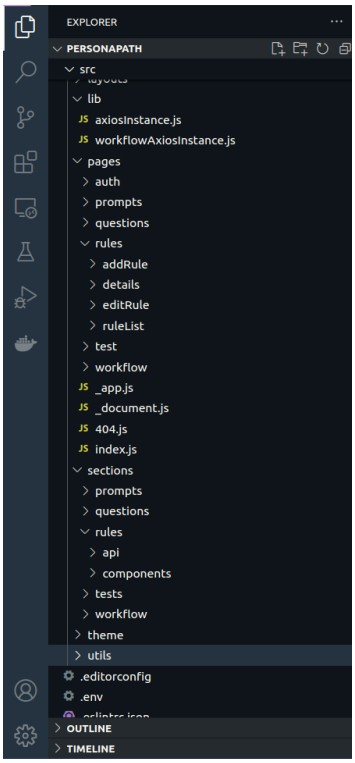
ساختار :

طراحی پروژه منطق کلی همه پروژه های Next.js ای را دنبال می کند، به طوری که سیستم از طریق پوشه pages انجام شده، پوشه ای مربوط به کامپوننت ها در نظر گرفته شده (sections) و فایل‌های پایه مربوط به درخواست های بک اند در پوشه ای مجزا به نام lib قرار گرفتند.

پوشه pages شامل پوشه های مختلف دیگر است که روت های اصلی برنامه را نشان می دهند. فولدر های تعریف شده عبارتند از test، rules، questions، prompts، auth و workflow که بخش‌های اصلی پروژه ما هستند. برای هر کدام از این مسیر ها در صورت نیاز سیستم مسیریابی داینامیک بر اساس id عنصر در نظر گرفته شده.

پوشه sections شامل فولد هایی از بخش‌های اصلی بیان شده است. هر کدام از این فولدر ها دارای بخش api و components هستند که به ترتیب درخواست های api بک اند هنگام استفاده انجام می دهند و کامپوننت های اشاره شده در pages را تعریف می کنند.

پوشه lib دارای instance های پروژه های بک اند هستند، به تعداد پروژه های استفاده شده instance داریم که در حال حاضر دو مورد هستند. AxiosInstance به پروژه اصلی بک اند اشاره می‌کند و اکثر درخواست ها در api های کامپوننت ها از آن استفاده می کنند. WorkflowAxiosInstance نیز به پروژه بک اند قسمت ورک فلو اشاره می‌کند و در api های قسمت مربوطه مورد استفاده قرار گرفته است. تصویر زیر نشان دهنده ساختار بیان شده است.



تصویر ساختار پروژه

توضیح ماژول ها:

بخش آزمون ها

بخش آزمون ها به عنوان یک بخش مرکزی برای سیستم پنل مدیریت عمل می کند و سازماندهی و مدیریت کارآمد آزمون ها را ارائه می کند. هنگامی که کاربران از طریق منوی ناوبری به صفحه اصلی که شامل آزمون هاست دسترسی پیدا کنند، لیست جامعی از تست هایی که در پنل تعریف شده اند به آنها نشان داده می شود. عملیات های امکان‌پذیر بر روی تست ها به شرح زیر است:

- **دریافت آزمون ها:**

پس از بارگذاری صفحه تست ها، سیستم به صورت پویا داده های تست را از پایگاه داده بازیابی می کند تا لیست آزمون ها را پر کند. این تضمین می کند که کاربران به به روزترین اطلاعات در مورد آزمون های موجود دسترسی دارند. فهرست آزمون ها یک نمای کلی ساختار یافته را ارائه می دهد که جزئیات مربوطه مانند عنوان هر آزمون و تاریخ ایجاد آن را نشان می دهد. علاوه بر این، دکمه‌های عملیات برای هر ردیف تست ارائه شده است که کاربران را قادر می‌سازد تا عملیات مختلفی مانند افزودن قوانین، ویرایش جزئیات تست یا حذف تست‌ها را انجام دهند.

منطق کلی کد:

- useEffect برای دریافت داده تست ها:

این کامپوننت از قلاب useEffect برای واکشی تمام داده های آزمون ها از سرور بر روی کامپوننت استفاده می کند. این تضمین می‌کند که لیست آزمون ها به صورت پویا با آخرین داده‌ها پر می‌شود و دیتا هنگام رندر کامپوننت ها به سرعت واکشی می شود.

- هندلرهای ویرایش تست، افزودن قانون و حذف تست:

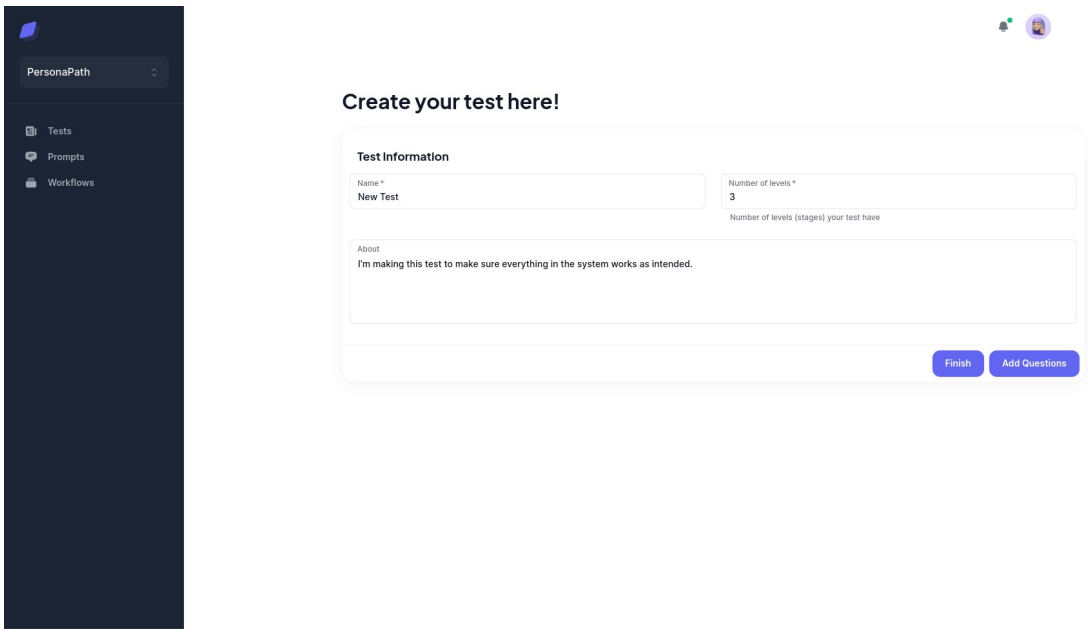
هندلرها (کنترل کننده های رویداد) برای مدیریت تعاملات کاربر با لیست تست ها پیاده سازی شده اند. این کنترل‌کننده‌ها اقداماتی مانند ویرایش آزمون، افزودن قوانین به تست‌ها و حذف تست‌ها از پنل را تسهیل می‌کنند. با تعریف این کنترل‌کننده‌ها، کامپوننت کاربران را قادر می‌سازد تا عملیات ضروری را به صورت یکپارچه در رابط لیست تست‌ها انجام دهند.

- بخش طراحی صفحه:

دستور بازگشت (return) این کامپوننت شامل بخش طراحی صفحه است که در آن عناصر UI با استفاده از کامپوننت های Material-UI ارائه می شوند. لیست تست‌ها به همراه اقدامات مرتبط مانند افزودن قوانین، ویرایش و حذف تست‌ها به صورت بصری جذاب و کاربرپسند به کاربران ارائه می‌شود. استفاده از مولفه‌های Material-UI جذابیت و عملکرد لیست تست‌ها را افزایش می‌دهد و به بهبود تجربه کاربری کمک می‌کند.

- **اضافه کردن آزمون:**

قسمت افزودن تست که با دکمه Add Test مورد دسترسی قرار می‌گیرد به کاربران اجازه می دهد تا آزمون های جدیدی را در پنل ایجاد کنند و آنها را قادر می سازد تا پارامترهای تست را بر اساس نیازهای خاص خود تعریف و پیکربندی کنند. هنگامی که کاربران اقدام به افزودن آزمون جدید کنند، فرمی به آنها ارائه می‌شود که می‌توانند جزئیات ضروری یعنی عنوان آزمون و تعداد سطوح شامل آن را و عناوین انتخابی مانند شرح مختصری درباره آزمون (about) را وارد کنند. همچنین دو دکمه برای این قسمت تعریف شده تا بهترین تجربه کاربری را برای کاربران ما رقم بزنند. دکمه اول (Finish) تست را ثبت کرده و به لیست آزمون ها باز می گردد، در حالی که دکمه دوم (Add Quesitons) تست را ثبت کرده و مستقیم کاربر را به صفحه افزودن سؤالات مربوط به تست انتقال می‌دهد تا کاربر مجبور به کلیک اضافه برای افزودن سؤالات نباشد و پس از ایجاد تست به راحتی بتواند به مرحله بعدی که اضافه کردن سؤالات است برود. این طراحی با هدف راحتی کاربران و ارائه بهترین تجربه ممکن به آن‌ها درنظر گرفته شده است.



صفحه اضافه کردن آزمون

منطق کلی کد:

- هندلر برای تغییر فرم:
  در این کامپوننت، یک هندلر (کنترل کننده) تغییر فرم برای گرفتن ورودی کاربر و به روز رسانی وضعیت فرم بر این اساس پیاده سازی می شود. این کنترل‌کننده تضمین می‌کند که تغییرات ایجاد شده در فیلدهای ورودی، همان لحظه در فرم منعکس می‌شوند و با پر کردن فیلد ها به کاربران بازخورد فوری ارائه می‌دهند.
- هندلر برای ارسال فرم:
  پس از تکمیل فرم ایجاد آزمون، کاربران می توانند داده های وارد شده را با استفاده از دکمه ثبت ارسال کنند. این دکمه فرآیند ارسال را آغار می‌کند و هندلر ارسال فرم را فراخوانی می‌کند که ورودی های فرم را تأیید می کند و داده های جمع آوری شده را برای ذخیره سازی به سرور ارسال می‌کند. همچنین پس از ثبت آزمون و بر اساس دکمه فشرده شده موقع ثبت تست کاربر را به قسمت مربوطه منتقل می کند.
- بخش طراحی صفحه:

در بخش آخر نیز عناصر UI برای تسهیل فرآیند ایجاد تست ارائه می شوند. این شامل فیلدهای ورودی برای عنوان آزمون، تعداد سطوح، و توضیحات، و همچنین دکمه‌هایی برای ارسال داده و تغییر مسیر بر اساس نیاز کاربر است. استفاده از مولفه های Material-UI یک رابط کاربری منسجم و بصری را تضمین می کند و تجربه کاربر را در طول ایجاد آزمون خوشایند می کند.

- **ویرایش آزمون:**

قسمت ویرایش آزمون کاربران را قادر می سازد تا جزئیات آزمون منتخب را تغییر داده و به روز کنند و به آنها اجازه می دهد پارامترهای آزمون را در صورت نیاز اصلاح و تنظیم کنند. هنگامی که کاربران ویرایش یک آزمون خاص را انتخاب می کنند، سیستم جزئیات تست منتخب را از پایگاه داده بازیابی می کند و فرم ویرایش را با اطلاعات موجود پر می کند. سپس کاربران می توانند قبل از ارسال داده های به روز شده، تغییرات لازم را در عنوان آزمون، تعداد سطوح یا توضیحات اعمال

کنند. صفحات ویرایش دقیقاً مانند صفحات اضافه کردن داده مربوطه هستند، به همین خاطر تصویر آن‌ها جدا قرار نمی گیرد.

منطق کلی کد:

- useEffect برای دریافت داده تست:

تمام کامپوننت ها از قلاب useEffect برای واکشی داده‌های مربوط به خود استفاده می کنند. در این کامپوننت داده تست منتخب از سرور دریافت شده و در فیلد های مربوطه قرار میگیرد . این تضمین می‌کند که آزمون به صورت پویا با آخرین داده‌ها پر می‌شود.

- هندلر برای تغییر فرم:

منطق این هندلر با کنترل کننده صفحه اضافه کردن تست یکسان است.

- هندلر برای ارسال فرم:

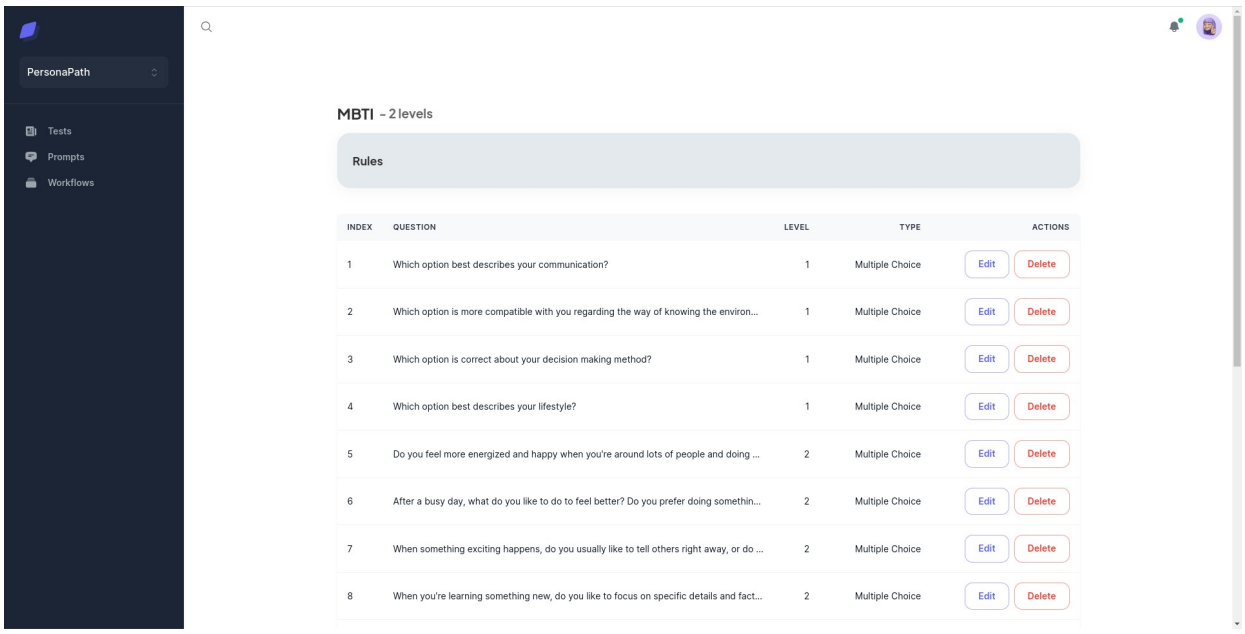
پس از تکمیل ویرایش آزمون، کاربران می توانند داده های وارد شده را با استفاده از دکمه Finish ارسال کنند. این دکمه پس از تأیید ورودی ها، داده های جمع آوری شده را به سرور ارسال می‌کند و کاربر و به صفحه قبلی یعنی لیست تست ها باز می گرداند.

- بخش طراحی صفحه:

طراحی این صفحه با صفحه افزودن تست یکسان است، با تفاوت دکمه ثبت که در ویرایش آزمون فقط یکی و با عنوان Finish است و همان منطق توضیح داده شده در افزودن تست را دنبال می کند.

- **جزئیات آزمون:**

با کلیک بر روی یک آزمون خاص از لیست تست ها، کاربران به صفحه جزئیات آزمون هدایت می شوند، جایی که می توانند اطلاعات جامعی درباره تست انتخاب شده مشاهده کنند. صفحه جزئیات آزمون یک نمای کلی از آزمون شامل عنوان، تعداد سطوح و متن «درباره تست» (در صورت وجود) در اختیار کاربران قرار می دهد. علاوه بر این، این صفحه دسترسی راحت به عملکردها و منابع مرتبط با آزمون را ارائه می دهد. از منابع و عملکرد ها می‌توان به همه عملیات های مربوط به سؤالات مانند مشاهده، ویرایش و حذف و دسترسی به بخش قوانین آزمون اشاره کرد.



منطق کلی کد:

- useEffect برای دریافت داده تست:

داده تست منتخب از سرور دریافت شده، سپس سؤالات مربوط به تست به صورت مجز از پایگاه داده دریافت می شوند .

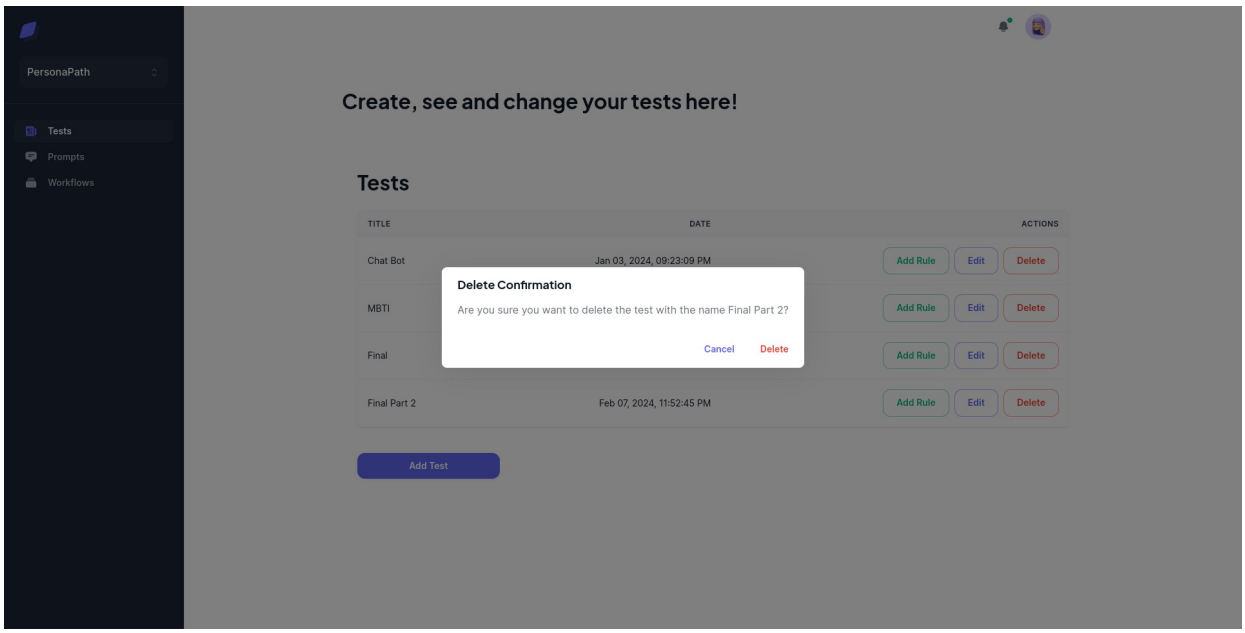
- بخش طراحی صفحه:

این صفحه نیز با بهره گیری از کامپوننت های MUI بهترین تجربه کاربری را برای افراد ایجاد می کند. علاوه بر جزئیات آزمون که در این صفحه نمایش داده می شوند، به دو کامپوننت مجزا دیگر که مربوط به لیست سؤالات و قسمت قوانین تست است اشاره می شود. با این ساختار کد ارائه شده مختصر و تمیز می‌ماند و از تکرار کد جلوگیری می‌شود. کامپوننت های اشاره شده در بخش‌های مربوطه توضیح داده خواهند شد.

درانتهای این کامپوننت نیز با دکمه Add Question به مسیر افزودن سؤالات مربوط به تست دسترسی همیشگی داریم تا به تعداد دلخواه سؤال به آزمون اضافه کنیم.

- **حذف آزمون:**

قابلیت حذف تست کاربران را قادر می سازد تا آزمون های ناخواسته یا منسوخ را از پنل حذف کنند و از مدیریت کارآمد منابع آزمون اطمینان حاصل کنند. هنگامی که کاربران حذف یک آزمون را انتخاب می کنند، سیستم از آنها می‌پرسد که آیا از حذف آزمون اطمینان دارند یا خیر و فقط با تأیید کاربر تست از سرور حذف می شود. این اقدام احتیاطی به جلوگیری از حذف‌های تصادفی کمک می‌کند و به کاربران این فرصت را می‌دهد تا قبل از ادامه اقدامات خود را تجدید نظر کنند.



مثال دیالوگ حذف داده

منطق کلی کد:

حذف داده نیازمند صفحه مجزا نیست و منطق کلی در هندلر کامپوننت دربرگیرنده داده تعریف می شود. توضیح زیر مربوط به کامپوننت دیالوگ فراخوانده شده درصورت استفاده از دکمه حذف است.

- بخش طراحی دیالوگ:

از آنجایی که منطق حذف در کامپوننت دربرگیرنده پیاده‌سازی می‌شود، مولفه دیالوگ سبک و طراحی شده و تنها خروجی بصری دیالوگ حذف را ارائه می دهد.

بخش سوالات

سؤالات اجزای جدایی ناپذیر مربوط به هر آزمون هستند و ابزاری را برای جمع آوری اطلاعات و بینش خاص از شرکت کنندگان در آزمون به کاربران پنل ارائه می دهند. برخلاف ویژگی‌های مستقلی که از طریق منوی پیمایش قابل دسترسی هستند، بخش سؤالات در چارچوب آزمون‌ها قابل دسترسی است که نشان دهنده رابطه جدانشدنی بین سؤالات و آزمون است، درواقع سؤالات متعلق به آزمون ها هستند و وجود هر یک بدون دیگری بی معناست.

کاربران می توانند با رفتن به صفحه جزئیات آزمون مورد نظر به بخش سؤالات دسترسی داشته باشند، جایی که می توانند سؤالات مربوط به آن آزمون را مشاهده، اضافه، ویرایش یا حذف کنند.

سؤالات دارای اجزای زیر هستند:

- **عنوان**: صورت سؤال را مشخص می کند. عنوان دو بخش انگلیسی و فارسی دارد، به طوری که بخش انگلیسی ضروری است و بخش فارسی انتخابی.
- **سطح:** هر سؤال به سطح خاصی از تست تعریف شده تعلق دارد و در همان سطح از کاربر تست دهنده پرسیده می شود.
- **نوع**: سؤالات سه نوع دارند و می‌توانند تشریحی، بازه ای یا چندگزینه ای باشند.
  - سؤالات تشریحی در چت بات از کاربر تست دهنده می‌خواهند پاسخشان به سؤال را تایپ کند.
  - سؤالات بازه ای به صورت درصدی پرسیده می‌شود به طوری که از فرد خواسته می‌شود موافقت یا مخالفتش را با سؤال طرح شده با درصد مورد نظر خود مشخص کند.
  - سؤالات چندگزینه ای رایج ترین نوع سؤال در سیستم ما هستند که برای فرد گزینه های ایجاد شده در طراحی سؤال را نمایش می‌دهند و از او خواسته می‌شود یک یا چند گزینه را انتخاب کند.
- **تعداد پاسخ‌های کاربر چت بات:** به تعداد پاسخ‌های مورد قبول برای انتخاب توسط کاربر چت بات اشاره می‌کند و تنها برای سؤالات چندگزینه ای مقدار می گیرد. این فیلد قرار گرفته تا به کاربران پنل این امکان را بدهد که در صورت نیاز و در سؤالات به‌خصوصی امکان انتخاب چندگزینه را به کاربر بدهند. این تعداد بیشترین تعداد گزینه ای که کاربر می‌تواند انتخاب کند را نشان می دهد.
- **گزینه ها:** به گزینه های سؤالات چندگزینه ای اشاره می‌کند و تنها برای چنین سؤالاتی امکان مقداردهی این فیلد وجود دارد. هر گزینه نیز می‌تواند متن انگلیسی و فارسی داشته باشد، به طوری که بخش انگلیسی ضروری و بخش فارسی انتخابی است.

اکنون که ساختار سؤالات را درک کردیم، به توضیح بخش‌های مختلف که سؤالات را دریافت، اضافه، ویرایش و حذف کرده‌اند می پردازیم:

- **دریافت سوالات:**

با بارگذاری صفحه جزئیات تست، سیستم به صورت پویا داده های تست را از پایگاه داده بازیابی می کند و سپس لیست سؤالات را دریافت می‌کند و به کامپوننت مربوطه ارسال می‌کند تا نمایش داده شود. فهرست، جزئیات ضروری هر سؤال را مانند عنوان انگلیسی، سطح، نوع و اقدامات ممکن را نشان می دهد و کاربران را قادر می سازد تا به طور موثر سوالات را مدیریت و سازماندهی کنند.

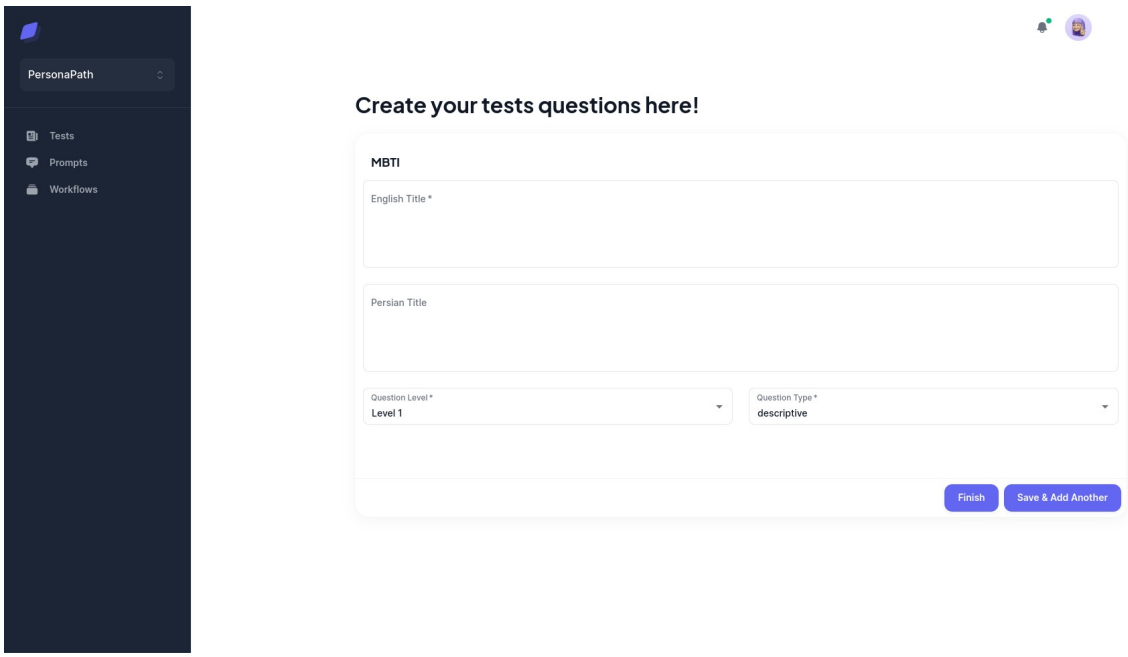
منطق کلی کد:

- تعریف عناوین نوع سؤال‌ها:
  نوع سؤال نمایش داده شده در لیست تبدیل عدد نوع به رشته توضیحی است، برای مثال نوع سؤال ۱ به سؤالات تشریحی اشاره می‌کند که خروجی نمایش داده شده به کاربر است.
- هندلرهای ویرایش سوال، حذف سؤال و جزئیات سوال:
  هندلرها برای مدیریت تعاملات کاربر با لیست سوالات پیاده سازی شده اند. این کنترل‌کننده‌ها اقدامات ویرایش سوال، حذف سؤال و نمایش جزئیات سؤال هنگام کلیک بر روی ردیف سوال را امکان پذیر می‌کنند.
- بخش طراحی صفحه:
  لیست سوالات به همراه اقدامات مرتبط با استفاده از مولفه‌هایMUI پیاده‌سازی شده است.

- **اضافه کردن سوال:**

قسمت افزودن سوال که با دکمه Add Question در صفحه جزئیات آزمون یا مستقیم از صفحه ایجاد تست مورد دسترسی قرار می‌گیرد به کاربران اجازه می دهد تا سوالات جدیدی را به آزمون خود اضافه کنند.

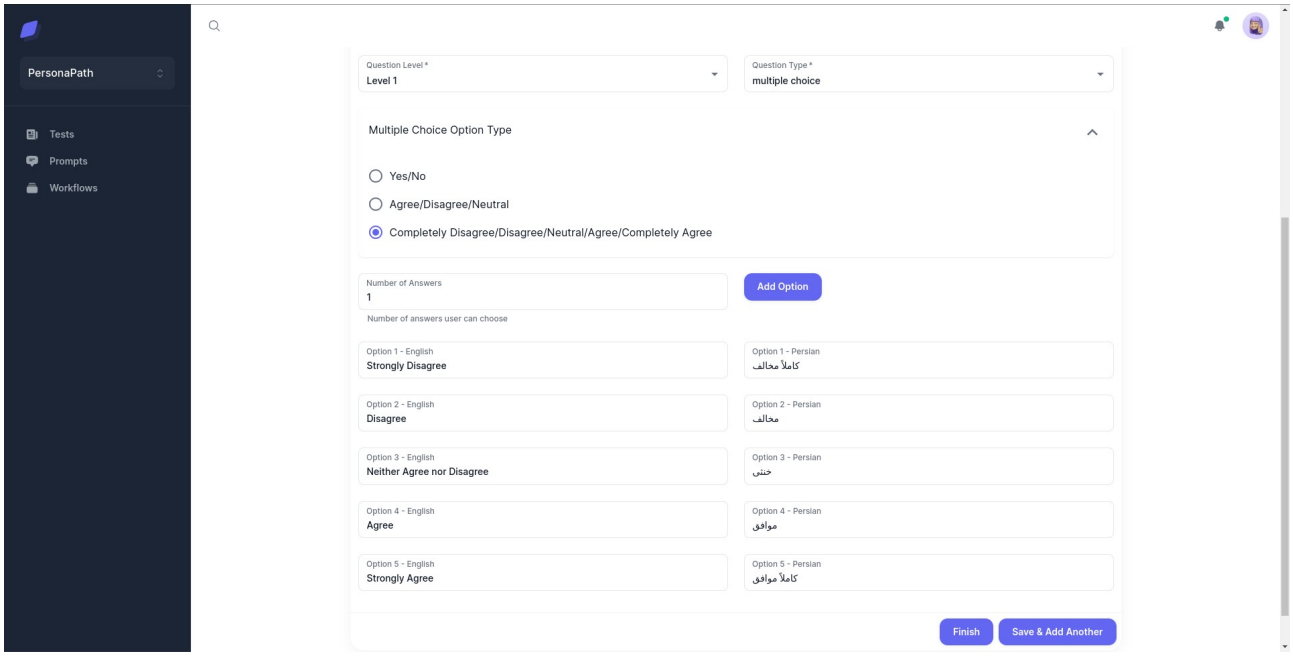
فرم افزودن سؤال بر اساس تغییر نوع (type) به فیلدهایش افزوده می‌شود یا از آن‌ها کاسته می شود. حالت اولیه تعریف سؤال روی تشریحی قرار دارد که دارای فیلد های عناوین (انگلیسی و فارسی)، سطح و نوع است. سؤالات نوع « بازه ای» نیز دارای فیلد های یکسان با سؤالات تشریحی هستند. در واقع نمایش سؤال در چت بات است که بر اساس نوع متفاوت است و داده‌های این دو حالت یکسان هستند.



صفحه افزودن سؤال با نوع سؤال تشریحی

در حالت چندگزینه ای تمام فیلد های اشاره شده یعنی عنوان انگلیسی، عنوان فارسی، سطح سوال، نوع، تعداد پاسخ‌های موردقبول برای انتخاب کاربر و گزینه ها که هر کدام علاوه بر حالت انگلسی حالت فارسی نیز دارند، در فرم حضور دارند. در بارگیری اولیه دو گزینه برای سؤال در نظر گرفته

شده است اما در صورت لزوم کاربران می‌توانند از دکمه اضافه کردن گزینه (Add Option) استفاده کنند تا تعداد گزینه ها را افزایش دهند. علاوه بر این قسمتی مربوط به حالت‌های مختلف گزینه ها در نظر گرفته شده تا کاربران در صورت نیاز به گزینه های تکراری ای مانند «بله/خیر»، «موافقم/خنثی/مخالفم» و «خیلی موافقم/موافقم/خنثی/مخالفم/خیلی مخالفم» مجبور به تایپ دوباره و دوباره آن‌ها نباشند. با کلیک بر روی قسمت Multiple Choice Option Type سه حالت ذکر شده برای گزینه ها نمایش داده می‌شوند و با انتخاب هر کدام گزینه های سؤالات به طور خودکار پر می‌شود تا بهترین تجربه کاربری را برای مصرف کنندگان رقم بزنند.
دو دکمه نیز درانتهای فرم برای این قسمت تعریف شده تا بهترین تجربه کاربری را رقم بزنند. دکمه اول (Finish) سوال را ثبت کرده و به لیست سوالات باز می گردد، در حالی که دکمه دوم (Save & Add Another) سوال را ثبت کرده و صفحه را دوباره بارگیری میکند تا فرم برای ثبت سؤالی دیگر آماده باشد. این دکمه ها طراحی شده اند تا کاربر مجبور به کلیک اضافه برای افرودن پشت سر هم سؤالات نباشد و وقت کمتری را صرف حرکت بین صفحات کند. این طراحی با هدف راحتی کاربران و ارائه بهترین تجربه ممکن به آن‌ها درنظر گرفته شده است.



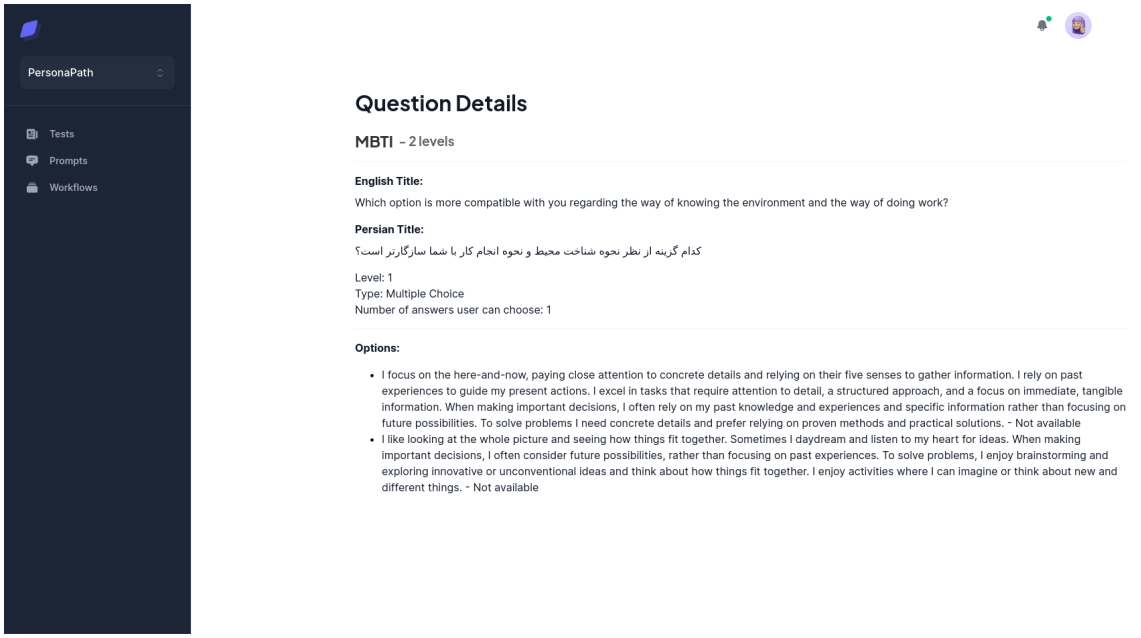
صفحه اضافه کردن سؤال چندگزینه ای

منطق کلی کد:

- تعریف عناوین نوع سؤال‌ها:
  همان طورکه توضیح داده شد، عدد مربوط به هر نوع سؤال به رشته قابل فهمی تبدیل می شود.
- هندلرها:
  هندلرها در این کامپوننت شامل کنترل کننده تغییرات فیلد ها، تغییرات حالت گزینه های از پیش تعریف شده، اضافه کردن گزینه و ثبت سؤال هستند.
- بخش طراحی صفحه:
  صفحه برای بهترین نمایش بصری grid بندی شده تا فیلد ها بتوانند به بهترین نحو کنار یکدیگر قرار بگیرند. همچنین شروطی روی بخشی از فیلد ها گزاشته شده که آن‌ها را بر اساس نوع سؤال نمایش می‌دهد یا پنهان می کند.

- **ویرایش سوال:**
قسمت ویرایش سوال کاربران را قادر می سازد تا پارامترهای موردنظر خود را در صورت نیاز اصلاح و تنظیم کنند. هنگامی که کاربران ویرایش یک سوال خاص را انتخاب می کنند، سیستم جزئیات آن را از پایگاه داده بازیابی می کند و فرم ویرایش را با اطلاعات موجود پر می کند. صفحه ویرایش دقیقاً مانند صفحه اضافه کردن داده مربوطه است به همین خاطر تصویر آن جدا قرار نمی گیرد.
منطق کلی کد:
  - useEffect برای دریافت داده سوال:
در یک هوک useEffect داده سوال منتخب از سرور دریافت شده و در فیلد های درست جای گذاری می شود .
  - هندلرها:
هندلرها دراین کامپوننت با کنترل کننده های اضافه کردن سؤال یکسان هستند.
  - بخش طراحی صفحه:
نمایش این کامپوننت با صفحه اضافه کردن سؤال یکسان است، با این تفاوت که تنها یک دکمه ثبت دارد (Finish) که در صورت کلیک تغییرات را ذخیره می‌کند و به صفحه قبلی (لیست سؤالات) باز می گردد.

- **جزئیات سوال:**
با کلیک بر روی یک سوال خاص از لیست، کاربران به صفحه جزئیات آن هدایت می شوند، جایی که می توانند اطلاعات جامعی درباره سوال انتخاب شده مشاهده کنند. صفحه جزئیات برای بررسی راحت سؤال با یک نگاه طراحی شده، به طوری که نکات پر اهمیت‌تر پررنگ تر نوشته‌اند و نکات کم اهمیت‌تر با اندازه متن معمولی پنل.



صفحه جزئیات سوال

منطق کلی کد:
  - useEffect برای دریافت داده سوال:

داده تست مربوط به سؤال و خود سؤال منتخب در هوک از سرور دریافت می شود.

- بخش طراحی صفحه:

این صفحه نیز با بهره گیری از کامپوننت های MUI بهترین تجربه کاربری را برای افراد ایجاد می‌کند و جزئیات آزمون و سؤال را نمایش می دهد.

- **حذف سوال:**

حذف سؤال نیز مانند حذف آزمون عمل می‌کند و با پرسش درباره اطمینان از حذف سؤال، آن را با درخواست مربوطه از سرور پاک می کند. ساختار کد نیز با حذف تست یکسان است.

بخش قوانین

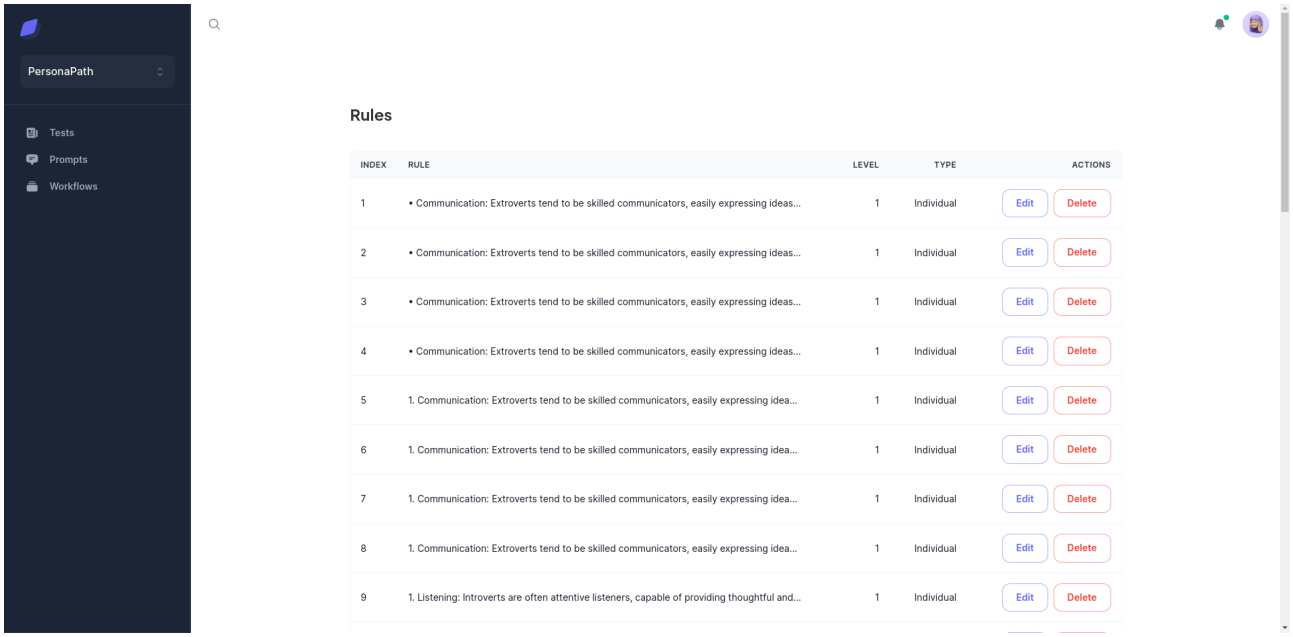
قوانین نیز مانند سوالات جزوی از یک آزمون هستند که به کاربران امکان تعریف شرایطی را می دهد که بر طبقه بندی آزمون دهندگان بر اساس پاسخ آنها به سؤالات خاص تأثیر گزارد. این قوانین نقش حیاتی در خودکارسازی فرآیند ارزیابی دارند و کاربران را قادر می‌سازند تا دسته‌ها یا نتایج از پیش تعریف‌شده را بر اساس پاسخ‌هایشان به افراد اختصاص دهند و در نتیجه تصمیم‌گیری و تحلیل را ساده‌تر کنند.
کاربران می توانند با رفتن به صفحه جزئیات آزمون مورد نظرشان و کلیک بر روی کارت مربوط به قوانین به این بخش دسترسی پیدا کنند.
قوانین دارای اجزای زیر هستند:

- **گزارشات:** نتیجه هر قانون یک گزارش است که می‌تواند هم به صورت انگلیسی و هم فارسی گزارش شود. طبق منطق قسمت‌های قبل در این بخش نیز گزارش انگلیسی الزامی و گزارش فارسی انتخابی است.
- **سطح:** هر قانون مربوط به سؤالات سطح به‌خصوصی است و سؤالات سطح مختلف نمی‌توانند با یکدیگر قانون بسازند.
- **نوع**: قوانین دو نوع دارند و می‌توانند فردی یا گروهی باشند.
  - قوانین فردی روی سؤالات تکی و گزینه های مربوط به آن سوال تعریف می‌شوند.
  - قوانین گروهی سؤالات را به دسته های مختلفی با تعداد مشخص شده تقسیم می‌کنند و بر اساس پاسخ تست دهنده به اجماع آن گروه از گزارشی تولید می کنند.
- **اندازه گروه:** فیلدی مربوط به تعداد سؤالات در هر گروه هنگامی که نوع قانون گروهی باشد تعریف شده است. این مقدار امکان ایجاد دسته های مختلف را به کاربران پنل می‌دهد تا انعطاف هر چه تمام تر را در تعریف قانون برای آزمون های خود داشته باشند.

اکنون که ساختار قوانین را درک کردیم، به توضیح بخش‌های مختلف که قوانین را دریافت، اضافه، ویرایش و حذف کرده‌اند می پردازیم:

- **دریافت قوانین:**

این کامپوننت تمام قوانین موجود مرتبط با تست انتخاب شده را از سرور بازیابی می کند و یک نمای کلی از پیکربندی قوانین را به کاربران ارائه می دهد. هر ردیف قانون ویژگی‌های ضروری را نشان می‌دهد، از جمله ابتدای گزارش انگلیسی قانون، سطح سؤالاتی که قانون روی آن اعمال می‌شود، نوع قانون (فردی یا گروهی)، و اقدامات مربوطه برای ویرایش یا حذف قانون. این ارائه ساختاریافته، ناوبری و مدیریت آسان داده‌های قوانین را تسهیل می‌کند، و کاربران را قادر می‌سازد تا به طور مؤثر بر معیارهای ارزیابی مبتنی بر قانون نظارت کنند.



صفحه لیست قوانین

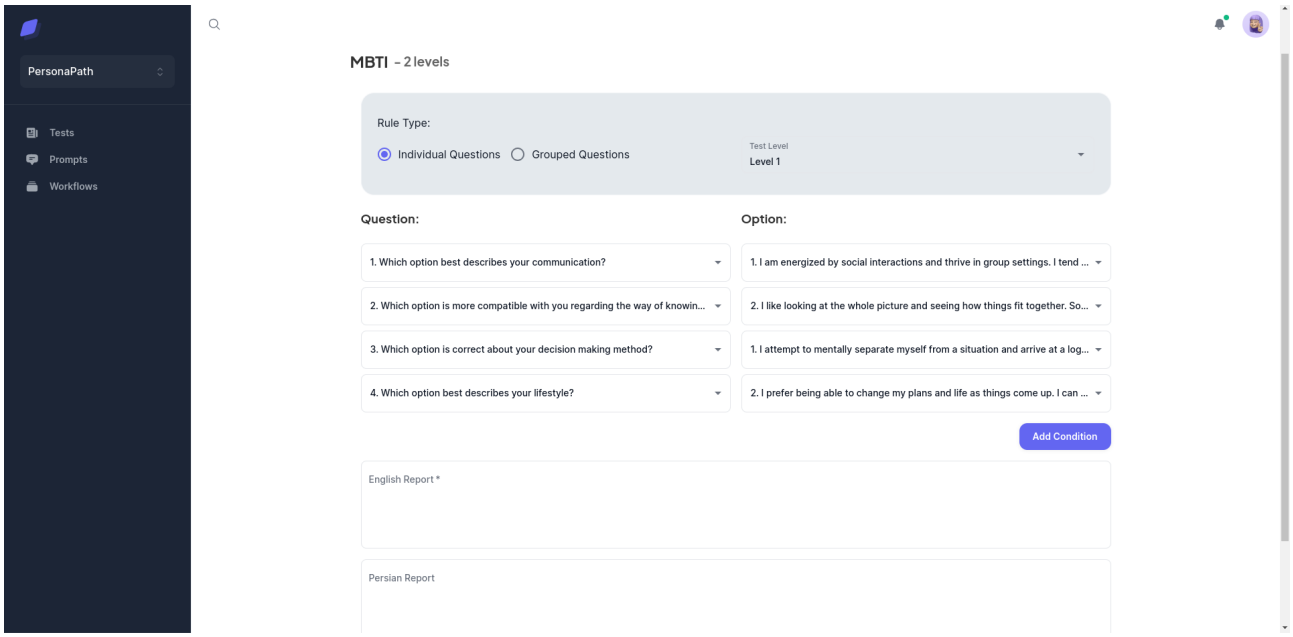
- **منطق کلی کد:**
  - useEffect برای دریافت داده قانون:
    در یک هوک با استفاده از آیدی آزمون منتخب اطلاعات همه قوانین مربوط به آن تست را دریافت می کنیم.
  - هندلرهای ویرایش قانون، حذف قانون و جزئیات قانون:
    هندلرها برای مدیریت تعاملات کاربر با لیست قوانین پیاده سازی شده اند. این کنترل‌کننده‌ها اقدامات ویرایش قانون، حذف قانون و نمایش جزئیات هر قانون هنگام کلیک بر روی ردیف را امکان پذیر می‌کنند.
  - بخش طراحی صفحه:

    ساختار لیست های بقیه صفحات دنبال شده و با استفاده از MUI کامپوننتی ساده و قابل فهم برای لیست قوانین ساخته شده است.

- **اضافه کردن قانون:**

قسمت افزودن قانون که با دکمه Add Rule از صفحه لیست آزمون ها یا لیست قوانین مورد دسترسی قرار می‌گیرد به کاربران اجازه می دهد تا قوانین جدیدی را به آزمون خود اضافه کنند.

فرم افزودن قانون بر اساس نوع (type) انتخاب شده تغییر می کند. حالت اولیه تعریف قانون فردی است که بین سؤالات مجزا شرط را قرار می دهد.

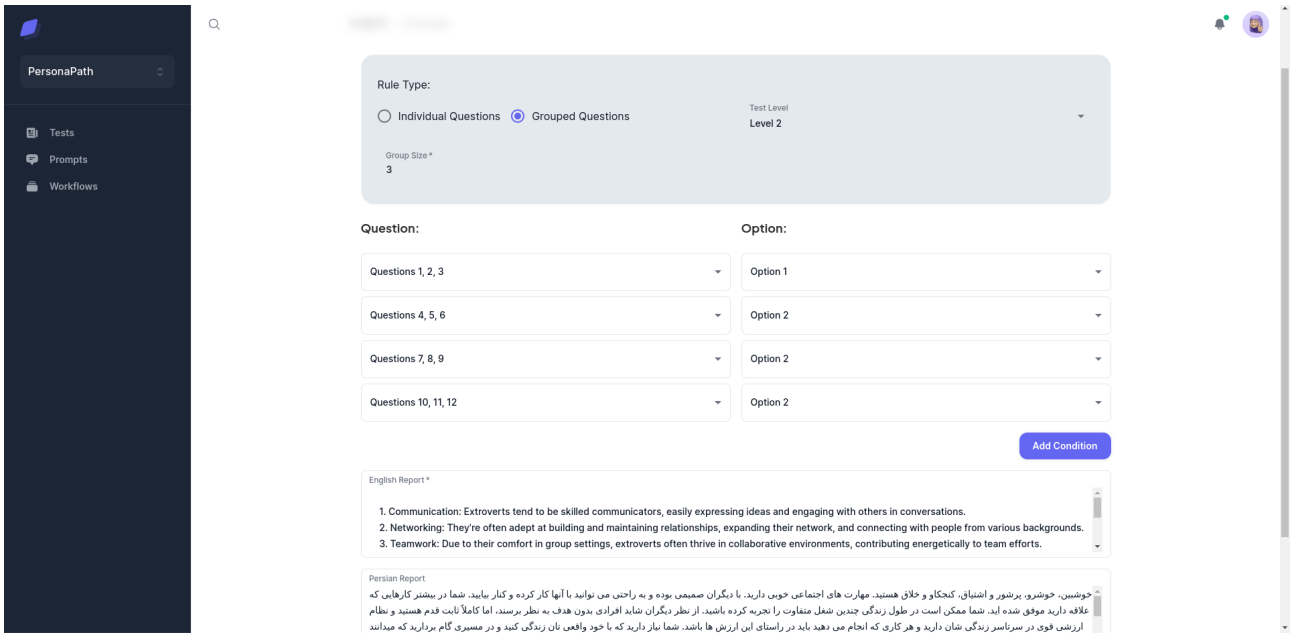


صفحه افزودن قانون با نوع فردی

در حالت گروهی سؤالات آن سطح از آزمون به دسته هایی با اندازه تعریف شده تقسیم می‌شوند و مینمم تعداد گزینه های سؤالات درون دسته به عنوان گزینه ها (گزینه ۱، گزینه ۲ و ...) برای سؤالات قرار میگیرند.

با دکمه Add Condition می‌توان به تعداد شرط ها اضافه کرد و قانون را خاص تر و خاص ترکرد.

دکمه Add Rule در انتهای فرم نیز برای ثبت قانون استفاده می‌شود و پس از ثبت، صفحه را برای افزودن قانون بعدی بارگیری دوباره می کند.



صفحه اضافه کردن قانون با نوع گروهی

منطق کلی کد:

- useEffect برای دریافت داده آزمون:
  قانون ها روی سؤالات تست ها قرار داده می شوند، از این جهت اولین کاری که در این کامپوننت انجام می‌شود دریافت داده‌های آزمون منتخب و سؤالات مربوط به آن است.
- useEffect برای کنترل داده ها:
  عناصر این کامپوننت بسیار وابسته به یکدیگر هستند و با تغییر یکی معمولاً پارامتر های دیگری نیز باید تغییر کنند. تغییرات این پارامتر های وابسته به هم در این useEffect اعمال می شود. پارامتر های وابسته به هم نامبرده عبارتند از سوالات، نوع قانون، تعداد سؤالات در گروه قانون و سطح سوالات. برای مثال با تغییر سطح سؤالات، پرواضح است که سؤالات بارگیری شده در صفحه نیز باید تغییر کنند،یا با تغییر اندازه گروه ها، سؤالات تقسیم شده در هر شرط باید تغییر کنند. این useEffect از مهم‌ترین عناصر کامپوننت های اضافه کردن و ویرایش قوانین است.
- هندلرها:
  این کامپوننت همچنین دارای تعداد زیادی هندلر است، برای مثال هندلر افزودن شرط، تغییر گزینه های سولات،تغییر سوالات، تغییر مقادیر نوع، اندازه گروه ها و سطح سؤالات و در انتها ثبت قانون که هر کدام وظیفه کنترل کردن بخشی از پارامتر های این کامپوننت را بر عهده دارند.
- بخش طراحی صفحه:
  ساختار این صفحه در هر دو حالت قانون یکسان است و تفاوت نوع قانون ها در داده نمایش داده شده در فیلد های شرایط است. ساختار طراحی شده با توجه به ماهیت پیچیده قوانین و در نظر گرفتن نهایت سادگی و سهولت استفاده طراحی شده است.

- **ویرایش قانون:**

قسمت ویرایش قانون کاربران را قادر می سازد تا پارامترهای موردنظر خود را در صورت نیاز اصلاح و تنظیم کنند. این صفحه از لحاظ بصری و منطقی شباهت بسیار زیادی به صفحه افزودن قانون دارد و از پارامترها و هندلر های یکسانی تشکیل شده است.

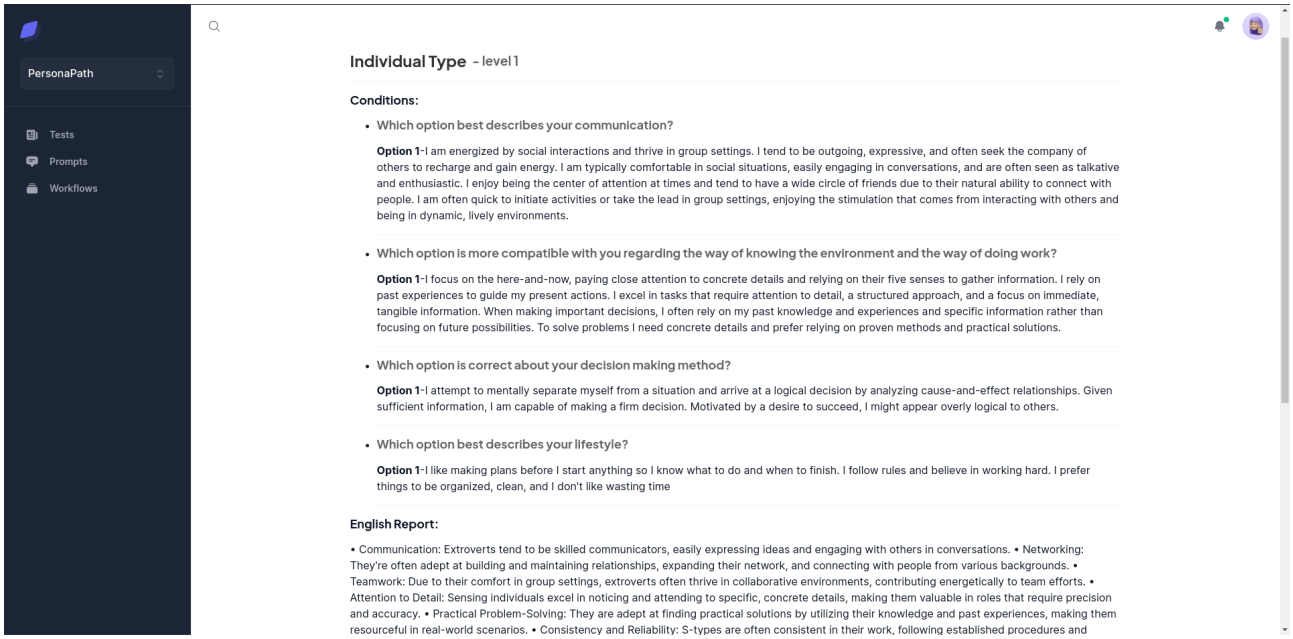
منطق کلی کد:

- useEffect برای دریافت داده:
  در یک هوک useEffect داده‌های آزمون دربرگیرنده، سؤالات آزمون و خود داده قانون از سرور دریافت می‌شود و در جای مناسب قرار می گیرد.
- هندلرها:
  هندلرها دراین کامپوننت با کنترل کننده های اضافه کردن قانون یکسان هستند.
- بخش طراحی صفحه:
  نمایش این کامپوننت با صفحه اضافه کردن قانون یکسان است و تنها فرق آن بارگیری داده قانون هنگام کلیک است.

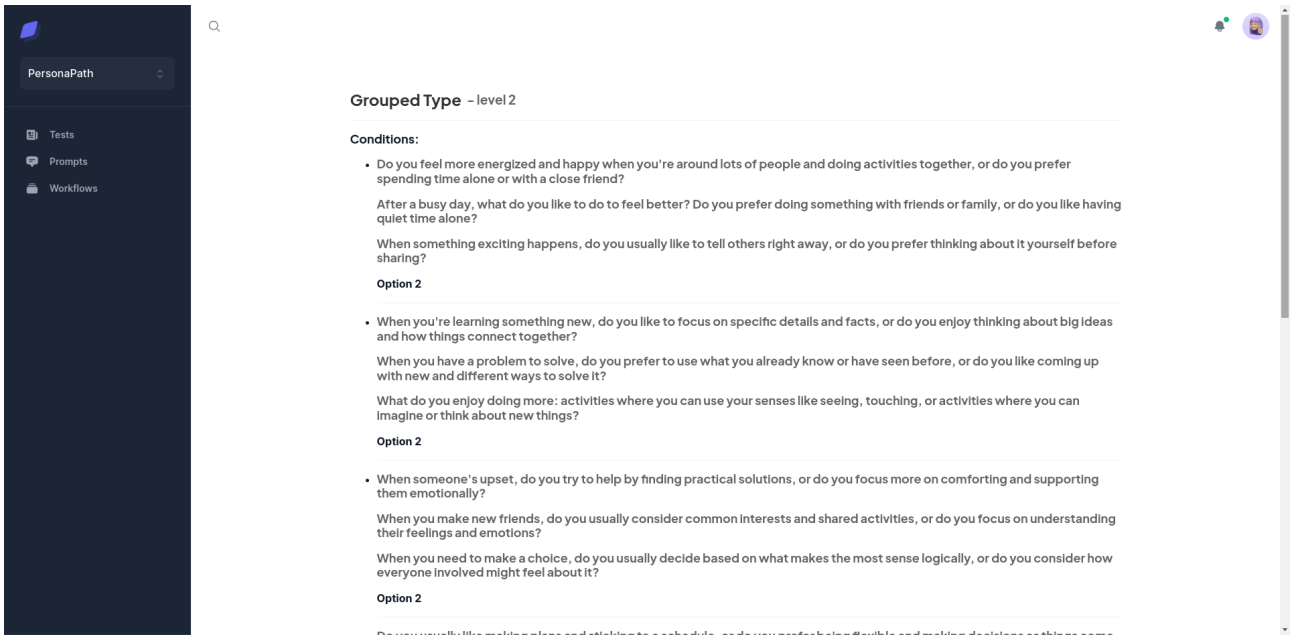
- **جزئیات قانون:**

با کلیک بر روی یک قانون خاص از لیست، کاربران به صفحه جزئیات آن هدایت می شوند، جایی که می توانند اطلاعات جامع قانون انتخاب شده را مشاهده کنند. صفحه جزئیات برای بررسی راحت

سؤال با یک نگاه طراحی شده، به طوری که نکات پر اهمیت‌تر پررنگ تر نوشته‌اند و نکات کم اهمیت‌تر با اندازه متن معمولی پنل. در این قسمت هم عملکرد و سادگی اولویت داده شده به طوری که اطلاعاتی که برای درک قانون سودمند بودند به صورت جزئی آورده شده‌اند ولی از نمایش اطلاعات اضافی مانند همه بخش‌های یک شرط (سوال) اجتناب شده است.
در بخش اولیه صفحه جزئیات نوع قانون و سطح سؤالات آن نمیاش داده شده است، سپس شروط تعریف شده که در قوانین فردی شامل یک سؤال و در قوانین گروهی شامل تعدادی سؤال هستند به نمایش گذاشته شده‌اند و در لنتها نیز گزارش های مربوط به هر قانون به زبان انگلیسی و فارسی (در صورت تعریف) آورده شده است.



صفحه جزئیات قانون فردی



صفحه جزئیات قانون گروهی

منطق کلی کد:

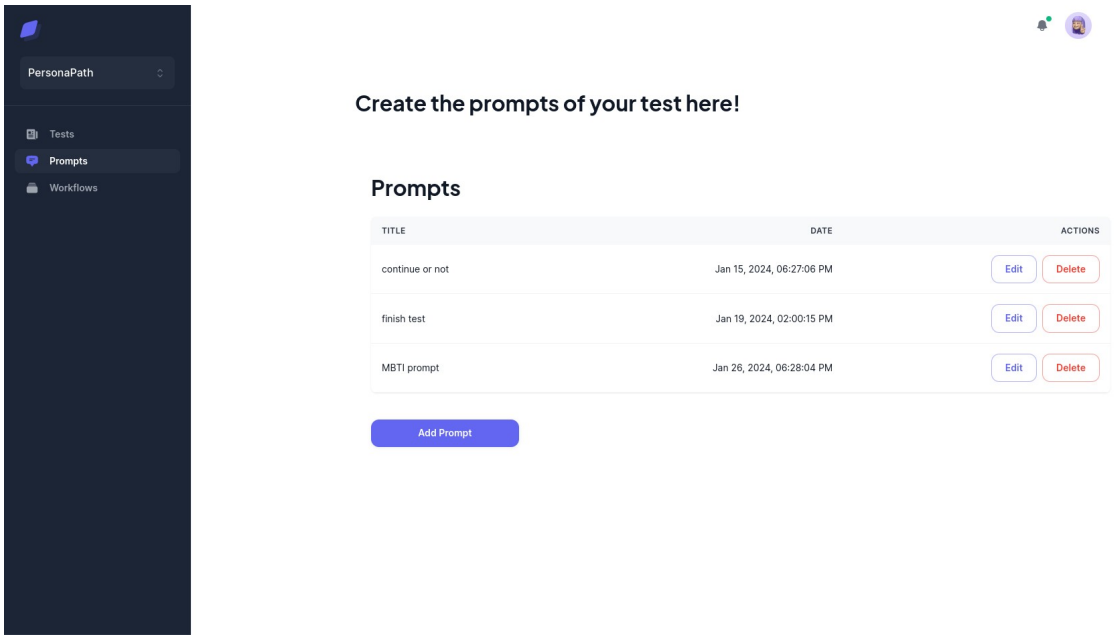
- useEffect برای دریافت داده:
  هر قانون دربرگیرنده سؤالات،گزینه و گزارش‌های خود است، به همین خاطر و بیش از حد شلوغ نکردن صفحه، داده‌های مربوط به آزمون و سؤالات آن بارگیری نشده‌اند.
- بخش طراحی صفحه:
  این صفحه با درنظرگرفتن سهولت بررسی قوانین که از نظر ماهیتی پیچیده هستند طراحی شده است و از نمایش داده‌های بیهوده برای کاربر اجتناب شده است.

- **حذف قانون:**
  حذف قانون نیز مانند حذف آزمون و سؤال عمل می‌کند و با پرسش درباره اطمینان از حذف، آن را با درخواست مربوطه از سرور پاک می کند. ساختار کد های حذف نیز با یکدیگر یکسان است تا جامعیت کد حفظ شود.

بخش درخواست ها

بخش درخواست ها (prompt) به کاربران این امکان را می‌دهد تا اعلان‌های مورد استفاده در تعامل با chatGPT را شخصی سازی کنند. با تعریف درخواست‌های سفارشی متناسب با زمینه‌ها یا اهداف خاص، کاربران می‌توانند اثربخشی و ارتباط با chatGPT را بهینه کنند و تبادلات آموزنده‌تری را با شرکت‌کنندگان در آزمون طراحی کنند.

- **دریافت درخواست ها:**
  این کامپوننت همه درخواست های موجود تعریف شده را بازیابی می‌کند و یک نمای کلی از پیکربندی پرامپت موجود را به کاربران ارائه می‌دهد. این نمای کلی شامل عنوان پرامپت، تاریخ ایجاد آن و عملیات های قابل انجام بر روی درخواست است.

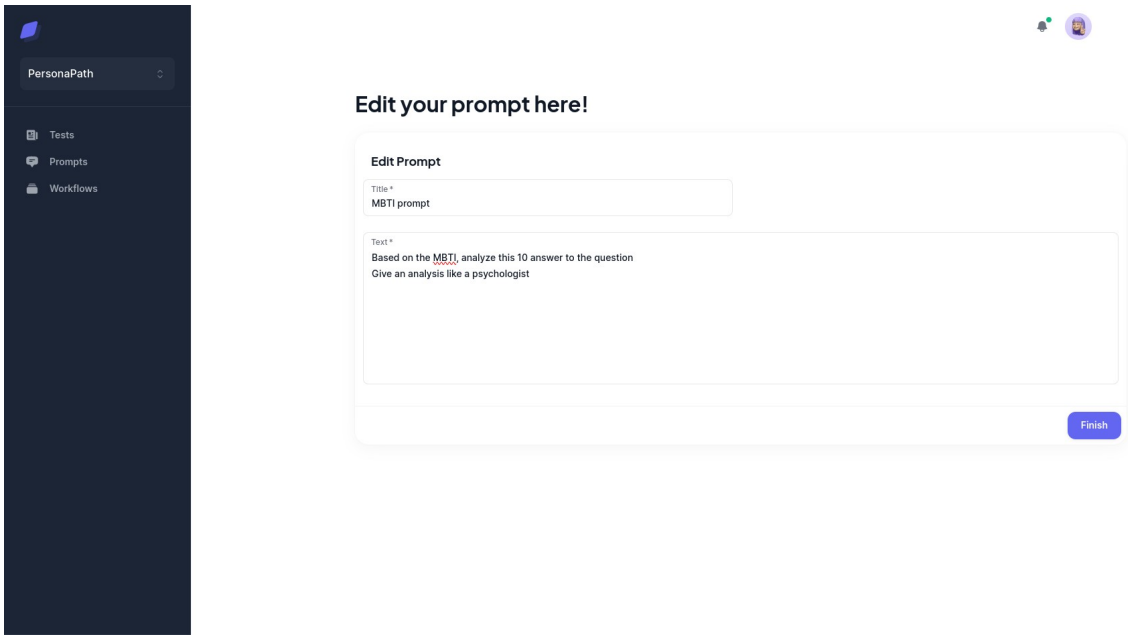


صفحه لیست درخواست ها

- **منطق کلی کد:**
  - useEffect برای دریافت داده:
    در یک هوک با استفاده همه پرامپت ها را دریافت می کنیم.
  - هندلرهای ویرایش، حذف و جزئیات:
    هندلرها برای مدیریت تعاملات کاربر با لیست درخواست ها پیاده سازی شده اند. این کنترل‌کننده‌ها اقدامات ویرایش پرامپت، حذف آن و نمایش جزئیات هر درخواست هنگام کلیک بر روی ردیف را امکان پذیر می‌کنند.
  - بخش طراحی صفحه:
    ساختار لیست های بقیه صفحات دنبال شده و با استفاده از MUI کامپوننتی ساده و قابل فهم برای لیست درخواست ها ساخته شده است.

- **اضافه کردن درخواست:**

قسمت افزودن قانون که با دکمه Add Prompt از صفحه لیست درخواست ها مورد دسترسی قرار می‌گیرد و به کاربران اجازه می دهد پرامپت جدیدی را به سیستم اضافه کنند.
پرامپت شامل یک عنوان و یک متن است. عنوان برای ساده سازی تشخص درخواست ها از هم در نظر گرفته شده است و متن درخواست نیز قسمتی از پرامپت است که به chatGPT فرستاده می‌شود.



صفحه افزودن درخواست

منطق کلی کد:

  - هندلرها:

کنترل کننده های تعریف شده در این کامپوننت تغییرات فیلد های اشاره شده را کنترل می‌کنند و در صورت کلیک دکمه اتمام داده را به سرور ارسال می کنند.

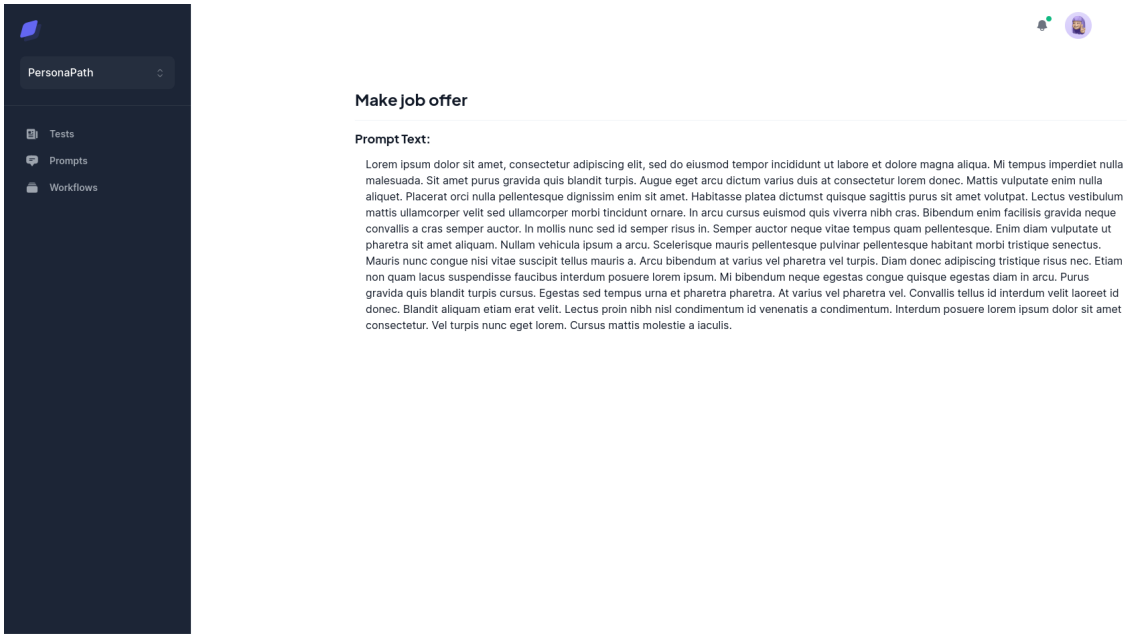
- بخش طراحی صفحه:
  ساختار این صفحه ساده در نظر گرفته شده است چرا که اصل و ماهیت پرامپت در متن درخواستی خلاصه می‌شود و افزودن ویژگی‌های دیگر فقط باعث شلوغی پایگاه داده و صفحات می شود.

- **ویرایش درخواست:**
  قسمت ویرایش درخواست به کاربران اجازه می‌دهد پرامپت های از پیش تعیین شده خود را بررسی دوباره کنند و در صورت نیاز تغییراتی که مدنظر دارند در آن اعمال کنند.

منطق کلی کد:

- useEffect برای دریافت داده:
  در یک هوک useEffect داده‌های پرامپت را از سرور دریافت کرده و در محل های مناسب برای ویرایش قرار می دهد.
- هندلرها:
  هندلرها دراین کامپوننت با کنترل کننده های اضافه کردن پرامپت یکسان هستند.
- بخش طراحی صفحه:
  نمایش این کامپوننت با صفحه اضافه کردن درخواست یکسان است و تنها فرق آن بارگیری داده پرامپت هنگام کلیک است.

- **جزئیات درخواست:**
  با کلیک بر روی یک درخواست خاص به صفحه جزئیات آن هدایت می‌شویم و می‌توانیم عنوان و متن قرار گرفته در پرامپت را مشاهده کنیم.



صفحه جزئیات یک درخواست

منطق کلی کد:

- useEffect برای دریافت داده:

  عنوان و متن پرامپت با استفاده از یک هوک useEffect دریافت می شود.

- بخش طراحی صفحه:

  این صفحه با درنظرگرفتن سهولت بررسی درخواست ها طراحی شده، به طوری که کاربر بتواند به راحتی درخواست را خوانده و در صورت نیاز از قسمت ویرایش تغییر دهد.

- **حذف درخواست:**

  حذف درخواست نیز مانند حذف آزمون، سؤال و قانون عمل می‌کند و با پرسش درباره اطمینان از حذف، آن را با ریکوئست مربوطه از سرور پاک می کند. ساختار کد های حذف نیز با یکدیگر یکسان است تا جامعیت کد حفظ شود.

بخش گردش کار

بخش گردش کار (workflow) به عنوان یک جزء محوری برای تعریف جریان توالی تعاملات و آزمون ها در چت بات عمل می کند و کاربران را قادر می سازد تا مسیرهایی پویا را برای مدیریت جریان کار و سفارشی سازی ارزیابی تنظیم کنند. در واقع گردش کار بیانگر چگونگی پیشروی در چت بات هنگام تست گیری است. با تعریف پله پله آزمون ها و درخواست ها، و شرط گزاری روی پاسخ تست دهنده در قدم‌های درخواست، سیستم به صورت کاملاً پویا عمل می‌کند تا بهترین تجربه کاربری را هم به کاربران پنل، هم به تست دهندگان و کاربران چت بات هدیه دهد.

گردش کار دارای اجزای زیر است

- **نام**: به هر گردش کار یک نام منحصر به فرد اختصاص داده می شود که به کاربران یک شناسه توصیفی برای مرجع و مدیریت آسان آن ارائه می دهد.
- **روال ها (routines)**: روال ها توالی کلی مراحل را در جریان کار نشان می دهند. این پارامتر شامل **قدم**های مختلف است.
- **قدم‌ها (steps)**: هر مرحله در یک روال به عنوان یک آزمون یا یک درخواست تعیین می شود.
  - **آزمون:** هنگامی که یک مرحله تست است، کاربران می توانند از لیست کشویی (dropdown) تمام تست های تعریف شده را مشاهده کنند و تست موردنظر خود را برای آن قدم انتخاب کنند. اگر آزمون انتخابی شامل چندین سطح باشد، یک کشویی دیگر ایجاد می‌شود که به کاربران اجازه می‌دهد تا سطح مورد نظر خود را برای قدم فعلی مشخص کنند.
  - **درخواست:** برای مراحلی که شامل درخواست ها (پرامپت ها) می‌شود، کاربران از فهرستی از درخواست های از پیش تعریف‌شده انتخاب می‌کنند. هر قدم درخواست همچنین دارای دو دراپ دون دیگر، با عناوین "اگر بله" و "اگر نه" است، که اقدامات بعد از این سؤال و جواب از تست دهنده را مشخص می کند. کاربران پنل منطق مسیریابی را بر اساس پاسخ های تست دهنده تعریف می کنند و روال بعدی را مشخص می کنند که پر صورت انتخاب آن انشعاب باید اجرا شود. اگر یکی از این دو فیلد خالی بماند، سیستم به طور خودکار درخواست تعیین شده را اجرا می کند و نتیجه را ایجاد می کند و از پیشرفت یکپارچه ارزیابی و تحویل بازخورد اطمینان حاصل می کند.

:

اکنون که ساختار گردش کار را درک کردیم، به توضیح بخش‌های مختلف که آن را دریافت، اضافه، ویرایش و حذف کرده‌اند می پردازیم:

- **دریافت گردش کار:**
  این کامپوننت تمام گردش کار های موجود در سیستم را به صورت لیست نمایش می دهد. اجزای این لیست عبارتند از عنوان گردش کار، تاریخ ایجاد آن و عملیات های قابل انجام برای هر ورک فلو که شامل ویرایش و حذف می شود.
  منطق کلی کد:
  - useEffect برای دریافت داده:

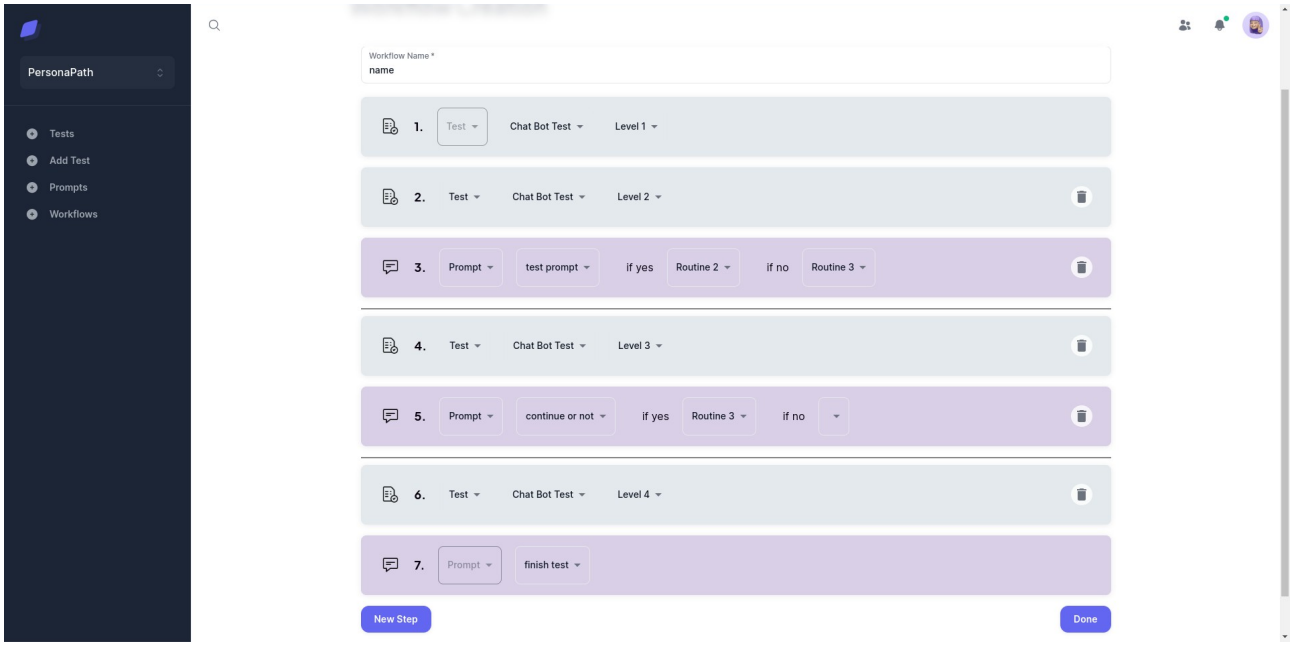
    در یک هوک useEffect همه ورک فلو های ایجاد شده دریافت می شوند.
  - هندلرهای ویرایش، حذف و جزئیات:
    هندلرهایی برای مدیریت تعاملات کاربر با لیست گردش کارها پیاده سازی شده اند. این کنترل‌کننده‌ها اقدامات ویرایش گردش کار، حذف آن و نمایش جزئیات هر ورک فلو هنگام کلیک بر روی ردیف را امکان پذیر می‌کنند.
  - بخش طراحی صفحه:

    ساختار لیست های بقیه صفحات دنبال شده و با استفاده از MUI کامپوننتی ساده و قابل فهم برای لیست ورک فلو ها ساخته شده است.

- **اضافه کردن گردش کار:**
  قسمت افزودن گردش کار که با دکمه Add Workflow در صفحه لیست ورک فلو ها مورد دسترسی قرار می‌گیرد به کاربران اجازه می دهد تا گردش کار جدیدی را به سیستم خود اضافه کنند.
  اولین فیلد موردنیاز در گردش کار نام آن است که بعداً به مدیران اجازه می‌دهد به راحتی ورک فلو های ایجاد شده را مدیریت کنند.
  گردش کار با طراحی گام به گام پیشروی کرده است، به طوری که هر قدم نشان دهنده یک آزمون یا یک درخواست است. این قدم‌ها هم با استفاده از آیکون های متفاوت، و هم با استفاده از رنگ‌های متفاوت از یکدیگر قابل تشخیص هستند.
  قدم اول هر گردش کار باید همیشه یک آزمون باشد و امکان تعریف پرامپت به عنوان اولین قدم وجود ندارد. بالعکس همین موضوع برای قدم آخر نیز صدق می‌کند،به طوری که قدم آخر تنها می‌تواند پرامپت باشد نه آزمون.
  همان‌طور که توضیح داده شد قدم‌های تست می‌توانند به سطوح مختلف آزمون نیز شکسته شوند. قدم‌های پرامپت هم به یک پرامپت اشاره می‌کنند و اینکه در صورت پاسخ بله یا خیر کاربر از کدام روال کار را سر بگیرند. این پرش ها روی روال ها انجام می‌شود نه روی قدم‌ها تا از مشکلات احتمالی نادیده گرفته شدن قدمی جلوگیری شود.

  با استفاده از دکمه Add Step می‌توان به قدم‌های یک گردش کار افزود و با کلیک روی آیکون سطل زباله سمت راست هر قدم نیز می‌توان آن را حدف کرد.

  دکمه Done نیز اتمام کار کاربر را مشخص می‌کند و درخواست ثبت را به سرور می‌فرستد و به لیست گردش کار ها باز می گردد.



صفحه افزودن یک گردش کار

منطق کلی کد:

- useEffect برای دریافت داده:

در ورک فلو به داده‌های آزمون ها و پرامپت ها نیاز داریم، این موارد را در یک useEffect قبل از بارگیری کامل صفحه دریافت می کنیم.

- useEffect برای کنترل داده ها:

علاوه بر هوک قبلی که برای دریافت داده ها استفاده شد، useEffect دیگری نیز در این کامپوننت وجود دارد که با تغییر هر قدم روال ها را تنظیم می کند تا همیشه ساختار مرتبط و درستی داشته باشیم.

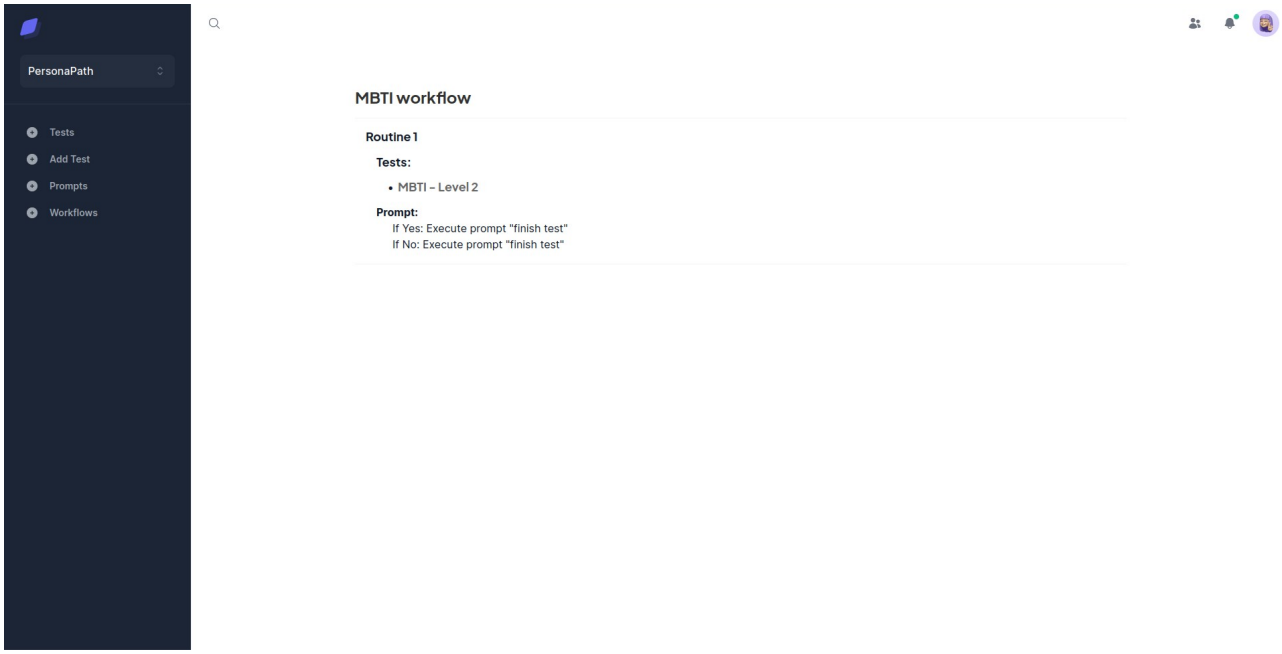
- هندلرها:
- در این کامپوننت هندلر هایی مانند هندلر های افرودن قدم و حذف آن، کنترل کننده های تغییر هر قدم از تست به پرامپت و بالعکس، هندلر داده ها مانند آزمون ها،درخواست ها، و شروط و تابع مربوط به ثبت داده را داریم.
- بخش طراحی صفحه:
ساختار این صفحه با درنظر گرفتن سادگی استفاده طراحی شده است، همچنین تلاش شده است تفاوت‌های بین قدم‌های آزمون و پراموت واضح و گویا باشد تا مشکلی برای کاربر ایجاد نکند.

- **ویرایش گردش کار:**
قسمت ویرایش ورک فلو کاربران را قادر می سازد تا پارامترهای موردنظر خود را در هر زمان اصلاح و تنظیم کنند. این صفحه از لحاظ بصری و منطقی شباهت بسیار زیادی به صفحه افزودن گردش کار دارد و از پارامترها و هندلر های یکسانی تشکیل شده است.

منطق کلی کد:

- useEffect برای دریافت داده:
  در ویرایش ورک فلو داده‌های مربوط به گردش کار منتخب را دریافت کرده، سپس داده های آزمون ها و پرامپت ها را دریافت می‌کنیم تا امکان تغییر داده اصلی را فراهم آوریم.
- useEffect برای کنترل داده‌ها و هندلر ها:
  کنترل کننده ها و هوک های استفاده شده همانند کامپوننت افزودن گردش کار است.
- بخش طراحی صفحه:
  ساختار این صفحه نیز با ساختار افزودن گردش کار یکسان است تا از مشکل سازی برای کاربران جلوگیری شود، طراحی دو نمایش متفاوت از داده‌های یکسان باعث آشفتگی پروژه می‌شود و تأثیر منفی روی کاربران می گزارد.

- **جزئیات گردش کار:**
  با کلیک بر روی یک گردش کار خاص از لیست، کاربران به صفحه جزئیات آن هدایت می شوند، جایی که می توانند اطلاعات جامع آن را مشاهده کنند.
- در صفحه جزئیات ابتدا نام گردش کار مشاهده می شود. سپس هر روال موجود در ورک فلو با جزئیات محدود نمایش داده می شود. آزمون های موجود در ورک فلو با نام و سطحشان نمایشس داده می‌شوند و درخواست های موجود نیز بر اساس مقادیر قرار داده شده در شروط پرامپت. اگر پرشی اتفاق افتاده باشد، شرط روال هدف پرش را نمایش می‌دهد و اگر شرط پرشی نباشد رشته ای گویای اینکه پرامپت انتخاب شده اجرا خواهد شد مشاهده می کنیم.



صفحه جزئیات گردش کار

منطق کلی کد:

- useEffect برای دریافت داده:
  برای نمایش جزئیات گردش کار ها به داده‌های آزمون ها و درخواست های مربوط به آن نیاز داریم،به همین خاطر علاوه بر داده خود ورک فلو، دیتا های آزمون های مرتبط و پرامپت های مرتبط نیز در هوک useEffect دریافت می شوند.
- بخش طراحی صفحه:
  این صفحه ساختار ورک فلو های تعریف شده را به وضوح نمایش می‌دهد. هر چه داخل پارامتر های ورک فلو می‌رویم، در صفحه نمایش هم با استفاده از یک tab این ساختار تو در تو را مشخص می کنیم.

- **حذف گردش کار:**
  حذف ورک فلو نیز نیز مانند حذف بقیه عناصر سیستم عمل می‌کند و با پرسش درباره اطمینان از حذف، آن را با درخواست مربوطه از سرور پاک می کند. ساختار کد های حذف نیز با یکدیگر یکسان است تا یکپارچگی کد حفظ شود.